



西北工业大学

本科毕业设计（论文）

题 目 面向微控制器平台的深度模型动态自适应压缩方法

专业名称 计算机科学与技术

学生姓名 马可

指导教师 刘思聪

毕业时间 2021 年 6 月

摘 要

随着深度模型在各个领域逐渐达成了超越人类的水平，模型本身的时间复杂度和空间复杂度也不断地突破上限，对运行设备的计算、存储等能力提出了越来越高的要求。在资源受限的移动或嵌入式等计算平台部署深度模型成为一种越来越重要的未来趋势。其中，微控制器以其小体积、低成本、低能耗等优势成为未来智能应用最为关键的部署设备之一，但其也因更加苛刻的极端资源条件（如MB级别以下的内存），极为显著地增加了深度模型的部署难度。此外，这些终端设备时刻都处于动态变化的平台资源情境（如存储资源、电量）和用户情境（如能耗、时延需求）中。但传统神经网络往往基于某种特定数据集和固定拓扑结构进行训练生成，缺乏一定的动态机制，对情境的自适应能力差。综上，在微控制器设备上成功部署深度模型，并实现其根据情境约束和需求进行精度和消耗的权衡折中，对于终端智能应用的发展具有重要意义。

针对上述挑战，本课题主要研究了深度模型于微控制器平台的部署与情境自适应问题，主要工作如下：

（1）提出一种基于压缩算子群的轻量化深度模型压缩方法，首先利用全局平均池化、模型结构同比例缩放思想等生成适配微控制器资源约束（如运行内存和持久性存储）的轻量化变体AlexNet模型。接着，利用“结构块”思想构建出基于压缩算子群的模型装载框架，进一步使用模型量化压缩技术完成本体模型和不同配置压缩算子于微控制器平台的成功部署，并通过三个生活中实际的基于图像分类的智能应用场景进行验证。

（2）提出一种软硬协同的情境（如平台资源约束、运行性能需求等）自适应框架，从应用模型层面的压缩算子群装载设计和硬件平台层面的电压、频率调整设计两个层面构建起策略搜索空间，使得模型所装载的智能应用从软、硬件维度进行运行方案选择，实现对当前情境更为多样化的最优适配。

实验结果表明，基于压缩算子群的轻量化深度模型压缩方法有效地实现了适配微控制器平台的深度模型部署。通过模型量化等技术在餐品分类任务中实现了50KB级别的模型生成，时延降低3倍以上，精度平均约70%且损失不超过3%。同时，这种软硬协同的情境自适应框架，具有相比于单独模型或平台层面设计更多的策略搜索空间。且两个层面的设计流程相对独立、互不影响，具有很高的可拓展性。

关键词：微控制器，模型压缩，情境自适应，软硬协同

ABSTRACT

As the deep learning model has gradually reached a level that surpasses human beings in various fields, the time complexity and space complexity of the model itself have continuously broken through the upper limit, and higher and higher requirements have been placed on the computing and storage capabilities of computing platforms. The deployment of deep learning models on resource-constrained mobile or embedded computing platforms has become an increasingly important future trend. Among them, microcontrollers have become one of the most critical deployment devices for future intelligent applications due to their small size, low cost, and low energy consumption. However, they also significantly increase the difficulty of deploying the deep learning model due to more extreme resource conditions, such as memory below the MB-level. In addition, these terminal devices are always in a dynamically changing platform resource context (such as storage resources, power) and user context (such as energy consumption, delay demand). However, traditional neural networks are often trained and generated based on a certain specific dataset and fixed topology, lacking a certain dynamic mechanism, and have poor adaptability to the context. In summary, the successful deployment of the deep learning model on the microcontrollers and the realization of the trade-off between accuracy and consumption based on contextual constraints and requirements are of great significance to the development of terminal intelligent applications.

In response to the above challenges, this subject mainly studies the deployment and context adaptation of deep learning models on microcontrollers. The main tasks are as follows:

(1) Propose a compact model structure and compression scheme design based on the compression operator group. First, we use the global average pooling and the concept of model structure scaling to generate a compact variant AlexNet which is able to meet microcontroller resource constraints (such as running memory and persistent storage). Next, we use the "block structure" idea to construct a model loading framework based on the compression operator group, and further use the model quantization compression technology to complete the successful deployment of the ontology model and its compression operators of different configurations on the microcontroller. Finally, the whole experiment is tested through three intelligent

applications in daily life based on image classification.

(2) Propose a context (such as platform resource constraints, performance requirements, etc.) adaption framework of software-hardware coordination, from the application(software)-level compression operator group loading design and the hardware-level voltage and frequency scaling design. As a result, the model loaded in intelligent applications can select running methods from the search design space above to achieve a more diversified optimal adaptation to the current context.

The experimental results show that the compact model structure and compression scheme design based on the compression operator group effectively realizes the deployment of the deep learning model which meets the microcontroller resource constraints. Through the technology of model quantization in the food classification task, a 50KB-level model is generated, the time delay is reduced by more than 3 times, and the accuracy is about 70% on average, of which loss does not exceed 3%. At the same time, this context adaption framework design of software-hardware coordination has a more diverse strategy search space than a single software-level or hardware-level design. Moreover, the design processes at the two levels are relatively independent and do not affect each other, so they have a high degree of extensibility.

KEY WORDS : microcontroller units, model compression, context adaptation, software-hardware coordination

目 录

摘 要.....	I
ABSTRACT.....	II
目 录.....	V
第一章 绪论.....	7
1.1 研究背景.....	7
1.2 微控制器与智能应用.....	8
1.3 研究目的及研究内容.....	11
1.4 本文内容安排.....	12
第二章 相关技术综述.....	15
2.1 引言.....	15
2.2 神经网络相关技术.....	16
2.2.1 模型压缩与加速技术.....	16
2.2.2 轻量化神经网络设计.....	19
2.2.3 平台资源自适应的神经网络架构搜索.....	20
2.2.4 动态神经网络推理.....	21
2.3 微控制器上的深度神经网络部署技术.....	23
2.3.1 面向微控制器的深度神经网络部署.....	23
2.3.2 深度神经网络推理过程中的存储/内存管理.....	25
2.3.3 实验平台及模型推理相关配置——STM32H747.....	27
2.4 基于微控制器的三种实际智能图像分类场景.....	29
第三章 基于压缩算子群的轻量化深度模型压缩方法.....	33
3.1 引言.....	33
3.2 模型结构和压缩方案设计流程.....	34
3.2.1 轻量化变体模型设计.....	37
3.2.2 基于压缩算子群的模型装载框架.....	39
3.2.3 模型量化压缩策略.....	44

3.3 实验结果展示.....	44
3.3.1 轻量化变体模型验证.....	45
3.3.2 基于多配置压缩算子群的模型装载效果.....	46
3.3.3 模型量化及平台部署.....	52
第四章 面向微控制器的软硬协同情境自适应框架.....	55
4.1 引言.....	55
4.2 软硬协同情境自适应框架建模.....	55
4.2.1 情境指标选择及其约束下的多目标优化搜索策略.....	56
4.2.2 软硬协同的自适应策略搜索空间设计.....	58
4.3 情境自适应框架实例验证.....	61
第五章 总结与展望.....	63
参考文献.....	65
致 谢.....	71
毕业设计小结.....	72

第一章 绪论

1.1 研究背景

近些年，随着深度神经网络（深度模型，或简称模型）领域及GPU等硬件技术的蓬勃发展，深度神经网络已逐渐在众多应用领域（例如语音识别^[1]、计算机视觉^[2]、自然语言处理^[3]等）和复杂系统设计（如自动驾驶^[4]、癌症检测^[5]、复杂游戏策略^[6]）中大放异彩，实现了领域领先的精度效果。同时，伴随领域内模型精度的不断突破，这些神经网络模型也逐渐在时间复杂度、空间复杂度等消耗指标上呈现了爆炸式的增长。例如，对一个224*224分辨率的彩色图像进行分类时，8层AlexNet具有0.61亿个网络参数，需要进行7.29亿次浮点型运算和花费240MB的内存存储；VGG-16^[7]更是达到了1.44亿个网络参数，150亿次浮点型运算和528MB内存花销的指标水平。总而言之，随着深度神经网络的性能增加，接踵而来的是模型深度、宽度的不断扩展，催生了深度神经网络高功耗、高存储的弊端。

不难看出，上述中神经网络消耗指标的增加，会严重影响深度神经网络在资源有限（体现于存储、计算、能耗、带宽等方面）的计算平台及应用环境中的进一步部署和运行。在如今追求万物互联、普适计算的时代背景下，越来越多的移动或嵌入式设备（如智能手机、无人机、机器人、智能眼镜等）需要进行深度神经网络的部署，实现智能增效。但它们不像具备强大计算、存储能力的传统CPU/GPU平台（如个人电脑、服务器主机等），无法满足这些深度学习模型在存储和运行上的需求。例如，以微控制器（Microcontroller Unit, MCU）这种资源极端受限的计算平台为例，主流的STM32系列（由ST公司生产研制的微控制器）往往只具备<1MB的内存使用空间（RAM）以及不超过2MB的持久性存储空间（Flash），不及VGG-16所需资源的百分之一。

在此背景下，科研和工业领域进行了大量关于神经网络轻量化部署的探索。模型压缩和加速技术（Model Compression and Acceleration）旨在不显著降低给定深度神经网络性能的前提下，对其进行剪枝、量化等压缩手段，使模型具备能够运行于资源受限设备的能力。此外，神经网络的设计研究者利用一系列启发式规则、计算模块化（Block-wise）等思想设计产生出本身具有相对低功耗、低存储的神经网络模型，满足对运行时资源条件的要求友好的同时也满足了较高的精度效果，例如SqueezeNet、MobileNet和ShuffleNet等。深度神经网络中前馈网络（Feed Forward Networks）的代表——卷积神经网络（Convolutional Neural Networks, CNN）是这些应用技术重点的研究对象，如图1.1。

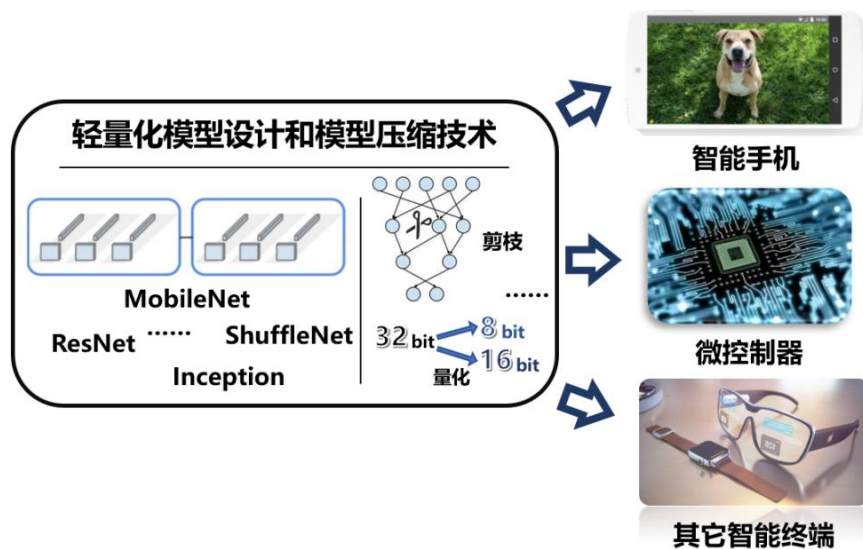


图1.1 轻量化模型部署技术

除了面向资源受限平台的深度神经网络压缩和模型设计，从实际应用出发，人们对于平台的神经网络应用也不再仅仅局限于“成功”部署的水平，而更向着“高效”、“灵活”部署的需求迈步，即深度神经网络不仅要满足平台的资源限制，更要尽可能地适应于平台资源情境或应用实际需求（如电量、存储资源、时延、能耗等），实现精度和消耗的巧妙权衡以及对情境的最优适配。基于此，平台资源自适应（Platform-aware）的神经网络架构搜索（Neural Architecture Search, NAS）、动态神经网络推理（Dynamic Neural Networks）等概念的提出和应用的落地为上述需求的实现带来了可能。相关工作将在第二章做更为详细地阐述。

综上，随着边缘智能、物联网相关技术的不断发展，在资源受限的移动端或嵌入式物联网设备上部署更加高效、灵活的深度神经网络，增效应用及设备智能水平，已然成为一项热门的研究领域以及一种不可阻挡的未来趋势。基于此，本课题则将研究重点放于微控制器上——一种相比于智能手机、树莓派等设备资源条件更为苛刻的计算平台。其以更低的成本、更小的体积、更泛在的使用性在众多智能应用中发挥着关键的作用，显著地影响着人们的生活。本章1.2节将对微控制器平台及基于微控制器的实际智能应用和前景挑战进行介绍、阐述。本章1.3节则将介绍此课题提出的“面向微控制器平台的深度模型动态自适应压缩方法”，并对其研究目的、意义和研究内容进行详细地描述。

1.2 微控制器与智能应用

微控制器（MCU），又称单片微型计算机（单片机），是一种将中央处理器CPU、随机存储器RAM、只读存储器ROM或闪存Flash、计数器Timer、USB、

A/D转换、多种I/O接口等等功能集成到一块硅片上而构成的一种小而完善的芯片级计算机，如图1.2。

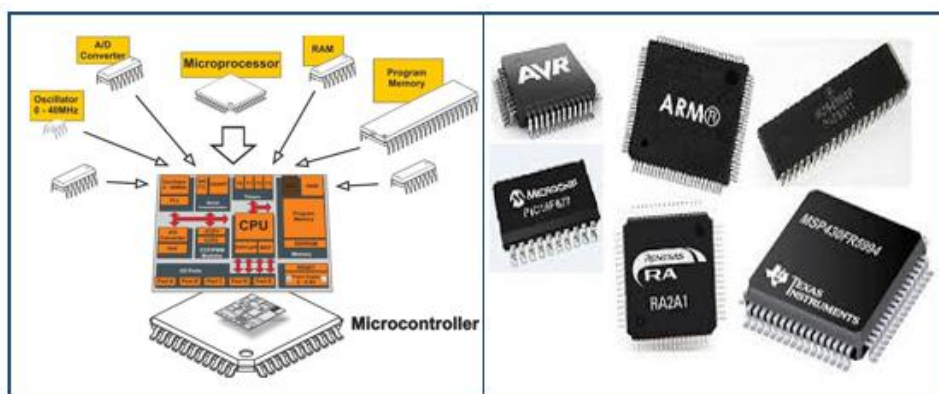


图1.2 微控制器抽象示意图（左）与真实外观（右）

文献^[8]称第一个微控制器是由TI的工程师Gary Boone 和 Michael Cochran研制的TMS 1000。后来随着ATMEL、Intel、ARM等企业的大量研发，从4位到32位、甚至到64位，从KB到MB，从几MHz到几百MHz，单片机在集成度、功能、速度、可靠性、应用领域及规格类型等全方位地向更高水平发展。如今的单片机主要还是基于冯·诺依曼（Von Neumann）结构的。这种结构清晰地定义了MCU的四大基本组成部分——CPU、读写存储器（RAM）和只读存储器（ROM or Flash）、一个或者多个定时/计时器以及I/O接口。

当前主流的MCU包括：a.由Intel推出的51单片机（一种兼容Intel 8051指令系统的单片机统称）；b.由TI研发的MSP430单片机；c.由Microship研发的PIC单片机；d.由ST推出的STM32系列单片机等。本次实验采用的是STM32系列中的STM32H747，其以ARM32位的Cortex-M7为内核，是当前功能最为强大、性能功耗比最高的MCU平台之一，更多规格说明将在第二章进行阐述。总而言之，在实际智能应用开发中，微控制器具有如下优势：

- ① 制造成本低。2018年，单片机的制造成本可低于0.1美元每单位，市场最便宜的8位微控制器售价低于0.03美元，同样数量的32位微控制器售价平均在1美元左右。
- ② 高集成度、高可靠性。作为一种微型计算机，微控制器将各功能部件集成在一块晶体芯片上，具有极高的系统集成程度和已知最小的计算平台体积。同时，微控制器所载芯片按照工业测控环境要求设计，内部布线远小于任何其它计算设备（如智能手机、个人电脑），程序、常数及表格固化在ROM或Flash中不易被破坏，信号通道均处于同一芯片上，故可靠性高。

- ③ 低电压、低功耗。当前市场主流的微控制器内部的工作电压均小于3.6V，而工作电流小于1A。

因此，虽然微控制器处理能力有限，在运行性能上远不及智能手机、个人电脑等移动式设备或传统CPU/GPU计算平台，但其以前述中极低的制作、使用成本和极高的性能价格比等优势，在设备数量大，尺寸、功耗限制要求高的众多智能应用领域中拥有着支配性的地位。同时，由此造成企业间强力的商业竞争，也催生了各种不同规格、不同成本、不同使用范围的微控制器类型，使得微控制器能够广泛使用于制造、军事、医疗、农业等领域。这里将对基于微控制器的智能应用开发及使用现状和前景挑战做简要阐述。

当前，深度模型增效的智能应用早已不局限于个人电脑或服务器主机等传统设备。实际生活中，在微控制器上运行日益复杂的深度模型，处理各种智能任务逐渐成为当代人工智能应用的基础和未来趋势。由此诞生的TinyML（微型机器学习）等概念也逐渐出现于科研和工业领域。文献记载，2020年基于微控制器的物联网设备数量已高达2500亿^[9]，涉及多个国家重大需求和民生领域。

- ① 智能制造：智能制造场景往往需要大量、高度集成的制造仪器和智能设备，因此拥有强烈性价比优势的微控制器具备广阔的市场。在制造部门，基于微控制器开发的智能应用可被用于零件故障检测——实时决策来停止由于设备故障造成的停机，流水线自动控制产品和设备——如机械臂、智能制造机床等。
- ② 智能交通：高集成度、高可靠性的微控制器智能应用可以通过传感器实时获取交通数据，更有效地确定交通路线，并减少紧急车辆的响应时间。如像Swim AI的企业就致力于在流媒体数据上使用基于微控制器的TinyML 应用来提高乘客的安全性，通过有效的路由来减少拥堵和排放。
- ③ 智能农业：农业场景下，微控制器同样以其小体积、高便携性等优势被广泛装载于家畜的可穿戴设备上。这些设备能够实时检测家畜的健康指标，如心率、血压、体温等，从而进行流行病检测和预防等工作。

除上述场景外，基于微控制器的微型智能应用还可以广泛存在于如下领域场景：零售业中的商品货架监控、个性化医疗中的智能医疗设备（如心脏起搏器）、公共安全中智能预警、安检搜查以及智能眼镜、智能手表、智能监控摄像头等生活中使用的各种智能物联网设备中。微型无人机、机器人等承载性低、体积小、低能耗需求高的移动无人设备更是以微控制器为主要的智能计算平台实现进一步的深度模型部署。

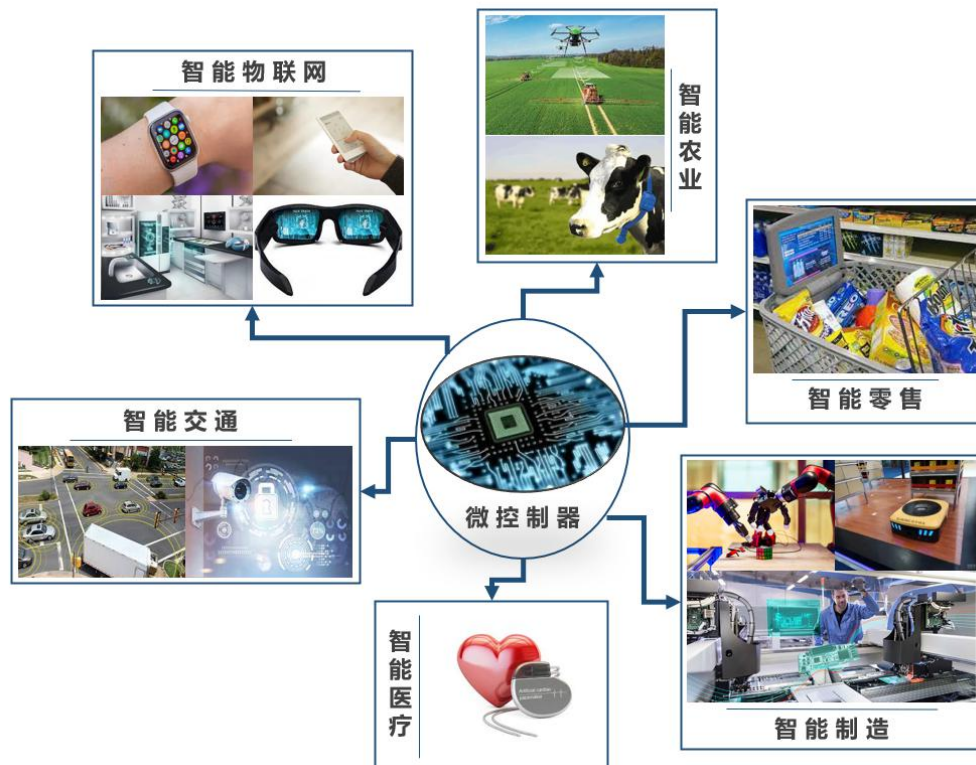


图1.3 微控制器支持的智能应用场景

综上，除了微控制器本身的独特优势之外，通过在微控制器这种微型设备上部署深度模型、进行设备的智能增效，人们能够在离数据生成端更近的地方进行数据分析和处理，进而使得人工智能的应用范围被大大扩展。但是，微控制器的资源预算非常有限，特别是在内存（SRAM）和持久性存储（ROM或Flash）方面，很大程度上限制了在其上部署的深度神经网络大小和性能。因此，相比于移动设备和服务器主机等平台，在微控制器上部署深度神经网络更具难度和挑战性（例如微控制器的MB级内存比智能手机小2~3个数量级），其中更多细节将在2.3相关技术综述中进行阐述。

1.3 研究目的及研究内容

随着深度模型在各个领域逐渐达成了超越人类的水平，模型本身的时间复杂度和空间复杂度也不断地突破上限，对运行设备的计算、存储等能力提出了越来越高的要求。一方面，仅依靠于追求硬件设备性能和资源上限的不断拓展是不符合实际的。另一方面，迎合普适计算、万物互联的时代需求，相比于在个人电脑或云服务器等传统CPU/GPU计算平台，在资源受限的移动或嵌入式等物联网终端部署深度神经网络，进行设备智能的增效，缩短智能应用与用户之间的距离从而更好地服务于用户，其本身更是一种至关重要的未来趋势但也更具挑战。其中，

微控制器以其小体积、低成本、低能耗、高扩展性、高集成度以及极大的泛用性成为未来智能应用最为关键的部署设备之一。但其也因更加苛刻的极端资源条件（普遍小于2MB的持久性存储和小于1MB的内存等），极为显著地增加了深度模型在其上部署和运行的难度。此外，另一项挑战是在实际应用中，这些计算平台都处于动态变化的平台资源情境中（如电量、存储资源），加之用户需求的因素，对应用加载的模型有着不同层面的约束和需求（如能耗、时延）。然而，传统神经网络往往基于某种特定数据集和固定的拓扑结构训练生成，缺乏一定的动态机制，从而更不具备自适应调优的能力。综上，在微控制器设备上成功部署深度模型，并使其具备自适应调优的能力，实现根据情境约束和需求进行精度和消耗的权衡折中和对情境的最优适配，对于终端智能应用的发展具有重要意义。

针对上述两项挑战，本课题首先探索了适配微控制器资源约束（如运行内存和持久性存储）的轻量化模型结构和压缩方案，利用“结构块”思想提出一种基于压缩算子群的模型装载框架，采用全局平均池化、模型量化等技术成功将深度模型及其压缩算子群部署于微控制器平台上，并通过三个生活中实际的基于图像分类的智能应用场景进行验证。接着，为了解决传统静态模型缺乏动态机制和调整能力的问题，本课题进一步提出一种软硬协同的情境（如平台资源约束、运行性能需求等）自适应框架，从应用模型层面的压缩算子群装载设计和硬件平台层面的电压、频率调整设计两个层面构建起策略搜索空间，使得模型所装载的智能应用从软、硬件维度进行运行方案选择，实现对当前情境更为多样化的最优适配。

1.4 本文内容安排

基于1.3研究内容的阐述，本课题的工作可以主要划分为如下两个模块：

（1）利用结构块思想提出一种基于压缩算子群的模型装载框架，综合利用轻量化模型设计和模型量化压缩等技术，将深度模型及其压缩算子群成功部署于资源极端受限的微控制器平台上。

（2）为了进一步实现模型所处的智能应用对于情境需求和约束的自适应调优，本课题提出一种软硬协同的情境自适应框架——在（1）基于压缩算子群的模型装载设计上结合微控制器平台层面的电压、频率调整技术，构建起更为多样化的自适应策略搜索空间，完成应用模型对当前情境的最优适配。

本文各章节安排如下：

第一章绪论，阐述了本课题的研究背景，对整个课题的部署平台——微控制器及其广泛的智能应用场景进行了简介，对本课题的研究目的、意义及研究内容进行了简要地说明。

第二章相关技术综述则对本课题相关的模型压缩和加速技术、轻量化神经网络设计、平台资源自适应的神经网络架构搜索和动态神经网络推理四个领域进行

阐述，并归纳总结了它们与本课题之间的联系和差异。接着，此章还对深度模型于微控制器上的推理框架、内存调度管理和本课题使用的STM32H747微控制器平台及相关推理配置进行了介绍。最后，此章节简介了可部署于微控制器上的三个生活中实际的图像分类智能应用场景，被用于本课题的实验验证。

第三章基于压缩算子群的轻量化深度模型压缩方法，首先选取了原始AlexNet模型作为课题的研究对象，结合微控制器平台资源限制（RAM、Flash等），对其结构特性和相关指标（模型相关指标和真实平台开销）进行分析，得到模型尺寸（由参数量决定）是深度神经网络于微控制器部署的最大制约。以此为指导机理，本课题通过全局平均池化和结构同比例缩放等完成轻量化变体AlexNet的设计。接着，为了实现后续模型自适应框架的搭建，本课题利用“结构块思想”通过前步骤生成的变体AlexNet和五种具备不同结构特性和运算模式的轻量化计算卷积块，提出一种基于压缩算子群的模型装载框架，并进一步使用模型量化压缩技术完成本体模型和不同配置压缩算子于微控制器平台的成功部署。

第四章面向微控制器的软硬协同情境自适应框架，首先对自适应的对象——情境做了定义和说明，并将情境自适应本质理解为策略搜索过程，从指标、空间、策略方面进行建模。所谓软硬协同情境自适应框架：“软”指在应用模型层面，利用前述基于压缩算子群的模型装载设计作为情境自适应的基础；“硬”则指在硬件平台层面，利用基于微控制器的电压频率调整技术增效模型自适应策略搜索空间。这种软硬协同的框架设计，具有相比于单独模型或平台层面设计更多样的策略空间，能够更多维地实现情境的最优适配，且两个层面的设计流程相对独立、互不影响，具有很高的可拓展性。

第五章总结与展望，则将对本课题工作进行全面的总结，并对其中存在的不足和未来改进的展望进行阐述。

第二章 相关技术综述

2.1 引言

在过去的几年中，深度神经网络已经拥有了显著的发展，在计算机视觉、语音识别、自然语言处理等领域取得了令人瞩目的成绩。随着这些深度神经网络精度效果的提升，网络模型的计算复杂度和资源消耗也在不断增加，为其于资源有限的移动或嵌入式设备上的部署构成了重大挑战。在此背景下，一系列模型压缩和加速技术以及轻量化的网络设计技术被提出。同时，要适应特定硬件平台的资源条件，不论是网络中精妙的结构块设计，还是模型深度、宽度及压缩方法的选择，这些过程涉及很多不同层面的超参数，需要专业经验和大量实验验证，无疑是一项繁琐的工作。因此，为了不依赖于人类，自动实现适应硬件的高效神经网络设计和部署，平台资源自适应的神经网络架构搜索技术成为最近几年研究的热点。此外，一种名为“动态神经网络推理”的理论研究从另一种机制层面赋予传统神经网络动态可变的可能性，使其能够在运行时对模型结构或参数进行自适应调整，以实现最优于当前输入样本的运算模式转化。值得注意的是，当前上述几项研究领域的理论和实验大多建立在计算机视觉中的图像分类任务上，故更多是以图像处理著称的卷积神经网络（Convolutional Neural Networks, CNN）作为重点研究对象。表2-1展示了部分主流的CNN模型以及它们拥有的参数量和所需要的计算量。

表2-1 主流CNN模型参数量及计算量

模型名称	参数量			计算量		
	尺寸(M)	卷积(%)	全连接(%)	Flops(G)	卷积(%)	全连接(%)
AlexNet	61	3.8	96.2	0.72	91.9	8.1
VGG-S	103	6.3	93.7	2.6	96.3	3.7
VGG16	138	10.6	89.4	15.5	99.2	0.8
GoogLeNet	6.9	85.1	14.9	1.6	99.9	0.1
ResNet-18	5.6	100	0	1.8	100	0
ResNet-50	12.2	100	0	3.8	100	0
ResNet-101	21.2	100	0	7.6	100	0

综合第一章对本课题背景、意义及内容的阐述，上述已有研究与本课题《面

向微控制器平台的深度模型动态自适应压缩方法》息息相关，却也拥有不同程度的差异。本章2.2节则将对模型压缩与加速技术、轻量化神经网络设计、平台资源自适应的神经网络架构搜索以及动态神经网络推理的已有研究分别进行简要阐述，并说明它们与本课题的联系和差异。本章2.3节则将从微控制器的实际部署出发，介绍深度神经网络于微控制器这种计算、存储资源极端受限的计算平台上运行所要具备的必要条件和最为关键的相关技术。本章2.4节则将对本课题使用的可部署于微控制器上的三个实际智能图像分类场景进行介绍。

2.2 深度神经网络相关技术

2.2.1 模型压缩与加速技术

模型压缩和加速技术旨在不显著降低模型精度的前提下，对模型采用剪枝、量化等等策略，减小深度神经网络模型存储和计算成本，使其能够成功部署于资源受限和具有严格时延需求的设备上。如今，国内外已有的模型压缩和加速技术研究可分为：参数剪枝、网络量化、低秩近似以及知识蒸馏。

（1）参数剪枝

参数剪枝（Parameters Pruning）基于神经网络具有不重要或不必要参数的假设，通过剔除这些冗余参数而保留对模型精度影响较大的参数，来实现模型参数量、计算量的有效缩减。当前工作以细粒度剪枝和过滤器级剪枝居多。细粒度剪枝（Fine-grained Pruning）方法以一种非结构化的方式来剔除参数，即卷积核中任何不重要的参数都可以进行剪枝。例如文献^[10]提出了一个动态网络手术框架，该框架由修剪和拼接两个操作组成——修剪用于剔除冗余参数，拼接用于恢复错误剔除的连接。他们的方法利用较少的训练次数获得了较好的压缩比。过滤器级剪枝（Filter-level Pruning）通过剪去卷积过滤器或通道（Channel），降低网络的宽度，属于结构化剪枝。这种方式对一层进行通道剪枝后，下一层的输入通道数也对应减少。Luo等^[11]提出了一种过滤器级剪枝方法——ThiNet。他们使用下一层的特征映射来指导当前层中的过滤器修剪，通过贪婪算法和最小化特征映射的重构误差来选择通道。相似的，He等^[12]提出了一种通过最小化特征映射误差来修剪过滤器的两步迭代算法，实现了VGG-16网络5倍的加速，且只有很小的精度下降。图2.1直观地展示了细粒度剪枝和过滤器级剪枝这两种方式。此外，结构化的剪枝还包括了组级（group-level）、向量级（Vector-level）、核级（Kernel-level）的剪枝方式。相比于非结构化剪枝，结构化的剪枝更受底层和硬件的支持。目前，参数剪枝是一种最为简单、有效和广泛利用的模型压缩方法。

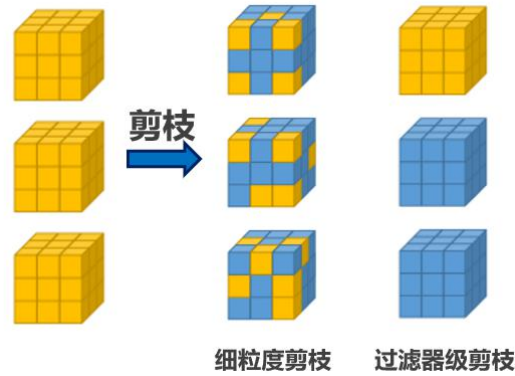


图2.1 细粒度剪枝和过滤器级剪枝示意图

（2）网络量化

网络量化指通过减少表示每个权重所需的比特数来压缩原始网络，实现模型复杂度降低的一类技术。传统神经网络的权重是以浮点数（32-bit）来进行存储的，通过将32-bit的权重量化压缩至不同比特的表示，模型参数空间被显著缩减。Gupta等人^[13]的工作在基于随机取整的CNN训练中使用了16位表示，显著减少了内存使用和浮点运算，且精度损失很小。Vanhoucke等人^[14]表明，在不显著降低分类精度的情况下，对参数进行8位量化可以显著加快速度。此外，还有将权重表示为1位的二值化（Binary）神经网络，例如BinaryConnect^[15]、XNOR^[16]等。其中XNOR不仅将权重进行二值化，更将网络的中间输出，即激活（Activations）进行二值化，参数量和计算量上分别实现了32倍和58倍的缩减，且在ImageNet数据集上只存在从75.7%到70.3%的精度损失。

（3）低秩近似

卷积层的本质就是通过一种 $w \in R^{w \times h \times c \times n}$ 四维张量的映射模式，实现输入到输出的表示和转换。低秩分解的动机就是寻找接近 w ，但更加有效的张量计算模式。根据将卷积层分解后的分量的多少，可以大致分为二分量分解、三分量分解和四分量分解。二分量分解即将权张量分为两个部分，用两个连续地层代替卷积层。文献^[17]通过将一个 $w * h$ 的卷积分别分解成 $w * 1$ 和 $1 * h$ 的张量，实现了文本字符识别数据集上4.5倍的加速，但只有1%的精度损失。此外，三分量分解一种简单的实现方式便是在二分量分解方法的基础上，进行两次连续的二分量分解。Zhang等^[18]对这种策略进行了验证，将卷积层分解成 $w * 1$ 、 $1 * h$ 和 $1 * 1$ ，发现在top-5精度降低0.3%的情况下，存在4倍的理论加速比。同样的，文献^[19]对AlexNet的第二层进行四分量分解，得到 $1 * 1$ 、 $w * 1$ 、 $1 * h$ 和 $1 * 1$ 的张量组合，同样实现了约1%精度损失的情况下，第二层4.5倍的加速。

（4）知识蒸馏

知识蒸馏不同于上述三种网络压缩或加速方式，其基于一种“教师——学生”

框架，并通过教师网络来训练学生网络。一般来说，因为教师网络是一个大型的深度神经网络或网络集合，而学生网络是一个紧凑、高效的网络且能够利用教师网络传递的更富信息的“知识”，所以学生网络能够获得比传统标签训练更高的准确率以及具备相比教师网络更加有效、轻量的网络拓扑结构，从而实现计算、存储成本的显著降低。例如文献^[20]提出FitNets，使用教师网络的softmax层输出作为知识去训练学生网络。Zagoruyko等提出注意力转移（Attention Transfer）的机制，使用原始特征图集（Feature Maps）中衍生出的注意力图（Attention Map）作为知识的表达，对学生网络的学习进行监督。

综上，2.2.1小节对于模型压缩和加速相关技术进行了简要的阐述。正如这些技术的实验结果所展示的一般，模型压缩和加速相关技术显著减少了深度神经网络计算和存储开销等，从而能够使模型更好地部署于资源受限平台。同时，在深度神经网络研究中，这些不同的技术和方法并非处于彼此完全独立的关系，而是能够兼用于同一模型之中，实现深度神经网络的共同增效。表2-2从模型压缩和加速技术的角度，非常简洁地归纳了这些工作。

表2-2 主流CNN模型参数量及计算量

压缩和加速方法	简介	细节对比
参数剪枝	剔除模型中不重要的冗余参数	简单、有效、超参数调整方案多 可以支持从头训练或使用微调训练
网络量化	减少表示模型权重、中间激活特征、输入输出的比特位宽	不影响模型拓扑特性 可以支持从头训练或使用微调训练
低秩近似	使用矩阵/张量分解近似传统卷积运算	简单、兼容性强 可以支持从头训练或使用微调训练
知识蒸馏	通过知识提取指导轻量化学生网络模型的训练学习	显著提高学生网络精度效果 只能支持从头训练

但是，当前这些模型压缩和加速技术也存在一定的不足，并不能完全解决本课题所要面临的问题和挑战。例如，大部分模型压缩和加速技术只使用深度神经网络本身的模型信息（如参数量、计算量）作为压缩方法的指导和有效性验证，并没有完全考虑真实硬件平台的性能指标（例如能耗、时延、内存/存储占用情况等）。而已有研究^[21]表明仅仅降低模型的参数量和计算量并非一定能使模型运行的能耗或时延降低。因此，这产生了只和模型相关的指标与真实平台性能、消耗指标之间的鸿沟，从而无法仅依靠实验结果完成实际模型在不同资源限制平台的部署。如何充分利用平台有限的资源条件，为这些平台设计特殊的压缩方法，仍然是一项开放式的难题。在此情况下，本课题将考虑深度神经网络于实际硬件

平台（微控制器）上的真实指标，通过真实平台指标让模型根据情境进行自适应压缩方案选择。

2.2.2 轻量化神经网络设计

随着Krizhevsky等人的工作在2012年ImageNet挑战赛中取得了突破性成果，一个包含61M参数的网络——AlexNet^[2]将人工智能研究学者的目光聚集起来，CNN进入快速发展的时期。随后，著名的VGGNet被Simonyan和Zisserman提出，通过显著加大模型的深度，在提高模型分类精度的同时，更将模型计算和存储开销提升至前所未有的高度（见图2.1）。因此，在资源有限平台部署情景下，深度神经网络的研究者们逐渐意识到不断加深、加宽模型来换取模型精度的提升，是一项不切实际的工作。且相关实验^[22]已证明——排除数据集过小带来的模型过拟合等问题后，一定阈值后网络深度的加大甚至还会降低分类的精确度（相对于较浅的网络而言）。

在上述背景下，越来越多轻量化的神经网络被设计出来。它们旨于通过研究者精妙的网络结构拓扑或利用块（Block）思想进行特殊的紧凑型卷积层的叠加等，直接设计出更加开销友好、轻量高效的深度神经网络，甚至在精度效果上更有所提升。所以轻量化神经网络设计其实本质也是模型压缩和加速技术的一种，不过其更加利用了学者的灵感和专家经验，生成了一系列样式各异，包含了不同设计智慧的深度神经网络。例如，ResNet的作者则想到了利用常规计算机视觉领域常用的Residual Representation的概念，并结合块叠加的思想，进一步将它应用在了CNN模型的构建当中，生成了构建起ResNet的基本残差学习块。ResNet通过使用多个有参层来学习输入输出之间的残差表示，而不同于一般CNN网络（如AlexNet、VGG等）那样使用有参层来直接尝试学习输入、输出之间的映射，如图2.2。实验表明使用一般意义上的有参层来直接学习残差比直接学习输入、输出间映射要容易得多（收敛速度更快），也有效得多（可通过使用更多的层来达到更高的分类精度）。

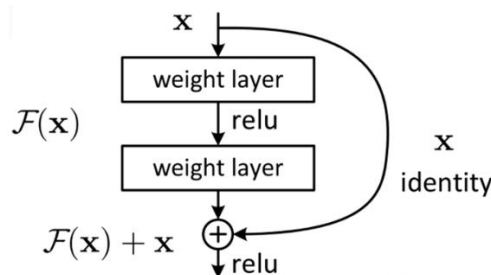


图2.2 ResNet残差块示意图

ResNet诞生后，更出现了越来越多不同的轻量化神经网络例如SqueezeNet^[23]设计一种Fire模块，通过将3*3的卷积替换成1*1的卷积和将3*3的卷积的通道数减少的方式，实现轻量化的模型设计；MobileNet^[24]采用一种深度级可分离卷积操作，将普通的卷积替换成Depthwise和Pointwise两种级别的可分离卷积，实现了计算量和模型参数量的大大降低；ShuffleNet^[25]在MobileNet的基础上提出组卷积和通道混洗的处理方法，通过实验验证说明其相比同样条件的MobileNet计算复杂度更低，精度效果更好。

综上，轻量化神经网络设计更专注于通过人类智慧直接设计出轻量高效的模型来降低模型开销和提高实际精度，并没有涉及到模型的“自适应”机制，更不能现在硬件平台上进行模型结构和运算模式的自动化调整。但这些网络中所涉及到的不同于传统卷积的紧凑型卷积层设计块，具备性能高、开销小的优势，启发了本课题的压缩算子群的设计。本课题正是借助这些设计智慧，利用上述部分轻量、高效的卷积算子构建出模型自适应的策略搜索空间的。

2.2.3 平台资源自适应的神经网络架构搜索

在2.2.1和2.2.2的工作中，研究学者提出了大量能够使得深度神经网络轻量化的模型压缩加速技术或神经网络架构设计。其中涉及到大量的超参数，需要人类专家利用充分的经验知识和实验测试来证明这些技术的有效性。这是一个耗时繁琐且容易出错的过程。因此，为了减少或去除掉模型设计中的人为工作，神经网络架构搜索技术在近些年吸引了大量的研究关注——其旨在通过构建搜索空间、搜索策略和性能评估策略，自动地设计出神经网络有效架构。

早期的神经网络架构搜索^[26-28]只关注模型架构的搜索空间，并没有将硬件平台的真实指标（如模型于计算平台上的运行时延、内存使用峰值等）考虑到架构搜索的过程中。自动生成的深度神经网络架构可以达到甚至超过人类手动设计的网络架构水平，但是由于搜索过程是硬件无关的，所以不能够保证硬件真实效率的提升。后来，具有硬件感知能力（Platform-aware 或 Hardware-aware）的神经网络架构搜索^[29-31]应运而生，旨在利用不同硬件平台的真实资源约束或指标反馈，为这些平台自动设计出特殊的最优深度神经网络架构。如图2.3。除了基于传统GPU或移动端设备的神经网络架构搜索，随着智能物联应用的拓展，领域也逐渐涌现出基于微控制器平台的神经网络架构搜索技术^[32-33]。同时，部分研究^[34-35]不仅探讨了硬件平台上的神经网络架构最优化设计，还实现了网络架构与硬件平台或推理框架的联合优化。例如，MCUNet^[9]提出一种基于微控制器的高效TinyNAS与轻量推理引擎TinyEngine联合设计的方案，并使用ImageNet级别的数据集进行推理验证。

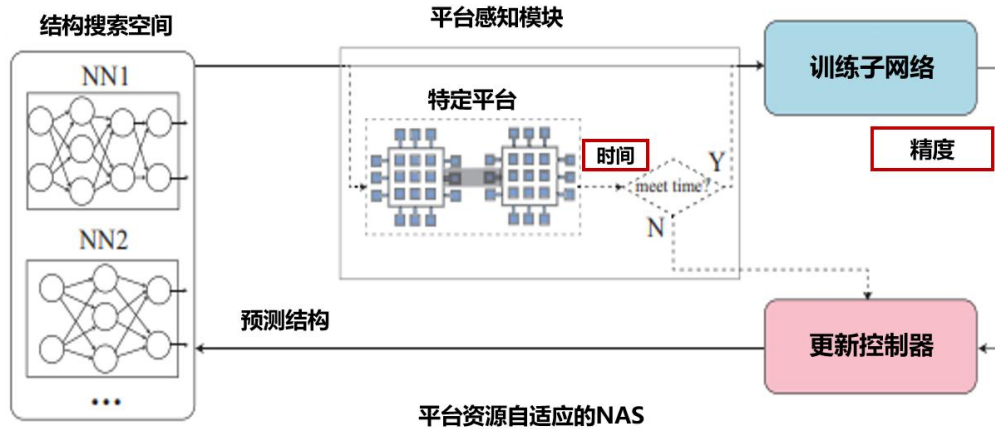


图2.3 硬件感知的神经网络架构搜索

具有硬件感知能力的NAS虽然实现了平台资源约束的最优化深度神经网络架构设计，但是相比于本课题所想要达到的深度神经网络自适应调优，依然存在如下差异：和2.2.2中的人工轻量化神经网络设计类似，NAS侧重于自动生成当前平台最优的神经网络架构，但无论是人工还是自动化设计，这些神经网络依然面临着计算图和网络参数一旦被选择和训练后就固定不变的问题，不具备网络结构或运算的动态变化机制，因而不适合用于运行时对深度神经网络进行动态自适应调优的设计。例如，对于某平台的最优架构生成：MnasNet需要花费至少40000个GPU小时；MCUNet需要花费至少300个GPU小时；Proxyless-NAS也需要花费至少200个GPU小时等。因此，本课题中的动态自适应压缩设计还需要从其它领域中寻找方案。

2.2.4 动态神经网络推理

在2.2.1到2.2.3小节中，不论是深度模型压缩和加速技术、轻量化神经网络设计还是解放人类设计的神经网络架构搜索技术，这些研究工作推动着神经网络向着更精确、更快速，开销更友好的方向发展，但其中的大多数网络都是以一种静态的方式进行着运算推理——计算图和网络参数在训练完成后就处于静态固定的状态。动态神经网络推理作为一门较为新兴的领域技术，相比于传统静态神经网络，能够对输入样本在推理阶段实现自适应的结构或参数调整，因而理论上具有更强的特征表达能力、适应能力和泛化能力等。

如一些研究^[36]通过建立一种门控机制实现深度模型中间层的自适应舍弃或选择性执行；BlockDrop^[37]另建立了一个策略网络，每个输入样例通过这个策略网络后生成骨干网络中每一层最终是否被执行的策略结果；BranchyNet^[38]使用基于置信度的方式决定推理过程是否需要早退出，并可以通过每一中间层新增的分

类器直接输出最终结果；SlimmableNets^[39]通过权重共享训练了一种可根据精度和速度要求自动调整模型宽度的网络。动态神经网络推理的研究不侧重于突破当前模型的精度，而是侧重于利用置信度、门控函数、策略网络等的动态机制实现深度神经网络对于输入样本的自适应调优，实现精度与开销的权衡。

但值得注意的是，当前动态神经网络推理的相关研究虽然实现了深度神经网络在推理过程中精度与开销的自适应调优，但是①当前深度学习推理框架和硬件大多是针对静态深度神经网络进行优化的，底层库或硬件平台（特别是资源极端受限的微控制器平台）缺乏对神经网络动态机制的支持（例如早退出、门控机制等），动态模型的实际运行时间也经常低于理论预测^[40]；②动态神经网络推理的当前工作主要围绕输入样本进行动态自适应的，其在硬件平台上根据资源情境和约束需求进行动态自适应的探索还处于非常萌芽的阶段。而其中的核心就是建立起情境指标和动态空间之间的映射机制。本课题《面向微控制器平台的深度模型动态自适应压缩方法》正是要探索：在压缩算子群动态空间确定后，利用情境与空间的映射机制，完成深度神经网络当前压缩方案的自适应选择和调整。

至此，2.2节对与本课题紧密相关的领域技术进行了简要地介绍和归纳，并于过程中阐述了它们和本课题之间的关联和差异。表2-3更加清晰明了地总结了上述内容。从它们的关联和差异还可看出，本课题①不止于关注平台无关的模型指标（如参数量、计算量），更关注于真实硬件指标（如时延、内存/存储占用情况等），以此指导模型在微控制器上的动态自适应框架部署；②具备模型动态自适应调优机制，使其能够根据情境约束和需求快速、低成本地选择当前最优的模型运行方案。

表2-3 本课题相关技术归纳

名称	简介	关联点	差异点
模型压缩和加速技术	不显著降低模型精度前提下，对模型采用参数剪枝、模型量化等技术	通过实验验证不同压缩方案和轻量化结构设计的有效性	缺少动态适应机制 缺少真实平台开销指导
轻量化神经网络设计	直接设计出高效、轻量的网络结构来提高性能和降低模型复杂度		
平台资源自适应的神经网络架构搜索	在搜索空间内，自动寻找和生成最适应于当前平台的神经网络架构	真实平台开销指标反馈	搜索时间长，不适应于本课题的动态机制搭建
动态神经网络推理	在推理时根据输入样本进行模型结构或参数的自适应调整	快速模型动态适应机制	局限于硬件/底层支持 适应对象不同

2.3 微控制器上的深度神经网络部署技术

本课题的研究平台是微控制器，其以极端有限的资源条件与其它主流的计算平台相比，如移动设备（如智能手机）和服务器主机等，在深度神经网络的部署上具备更大的难度和挑战。因而2.3节将对当前微控制器上的深度神经网络部署相关技术和理论做一定的介绍和阐述。2.3.1将对微控制器的配置参数、模型部署流程及神经网络推理框架进行介绍。2.3.2将对微控制器上神经网络推理过程中最重要的存储使用和内存调度情况进行总结和梳理。2.3.3则将对本课题实际使用的微控制器平台——STM32H747进行简要的介绍。

2.3.1 面向微控制器的深度神经网络部署

微控制器是一种微型计算机，配有低频处理器、持久存储器（ROM或Flash）和静态内存（SRAM）等，且都存在于一个芯片之上。相比于传统CPU/GPU设备（如智能手机、个人电脑、云服务器等），微控制器价格低廉且极为节能，但在具备高集成度特性的同时，资源预算也十分有限：文献^[9]提到微控制器的片上（On-chip）存储器比移动设备（如智能手机）小3个数量级，比云GPU服务器小5-6个数量级，因而在微控制器上部署深度神经网络极其困难。图2.4对比了主流的三种神经网络的期望内存/存储和ARM Cortex-M7微控制器（STM32H747）、云服务器（NVIDIA V100）、智能手机（MI 11）的硬件容量。由图可见，即使是8-bit量化版本的以轻量化著称的MobileNetV2^[41]，所需内存也是STM32H747微控制器拥有内存容量的3.4倍。

	云服务器 (NVIDIA V100)	智能手机 (MI 11)	微控制器 (STM32H747)		ResNet-50	MobileNetV2	MobileNetV2 (8-bit)
Memory	16GB	8GB	512KB	← gap →	7.2MB	6.8MB	1.7MB
Storage	TB~PB	>128GB	2MB	← gap →	102MB	13.6MB	3.4MB

图2.4 不同硬件平台容量和主流神经网络期望消耗

除了存储/内存方面，微控制器作为一种“裸设备”，不具有完备的操作系统，更不存在Pytorch、TensorFlow等当前主流的神经网络训练框架，因而更不可能在微控制器平台进行深度神经网络的训练。所以不论是传统静态神经网络还是本课题涉及到的基于压缩算子群的动态自适应网络模型，它们的训练必须在部署到微控制器之前完成。此外，微控制器同样也不具备Python解释器或C语言编译器等IDE，不具备实时编写代码并运行网络推理的能力，而从训练出来的神经网络模型到真正部署于微控制器上，更需要当前一些面向微控制器平台的神经网络推理

框架的支持。此小节接下来的部分将主要针对STM32系列微控制器平台，对神经网络部署的整个流程及当前主流的推理框架进行介绍。

微控制器上的神经网络部署涉及到训练、转换推理、应用部署三个流程。在训练部分，由于当前微控制器还不具备模型训练的能力，神经网络的训练依旧依赖于传统CPU/GPU计算平台以及TensorFlow、Pytorch、Caffe等训练框架，此处不对这些框架进行详述。而对于一个已训练好的模型，将之运行于多种多样的设备上，其中就需要推理框架（Inference Framework）的支持，如图2.5所示。而针对不同平台，更大硬件厂商都相应推出了自己的推理框架比如Intel的OpenVINO，ARM的ARM NN，NV的TensorRT等，同时还有针对移动端神经网络推理的NCNN和TensorFlow-Lite等。TVM则更是通过一种深度学习编译器的技术，引入中间表示，显著增加了不同平台推理框架的可移植性。

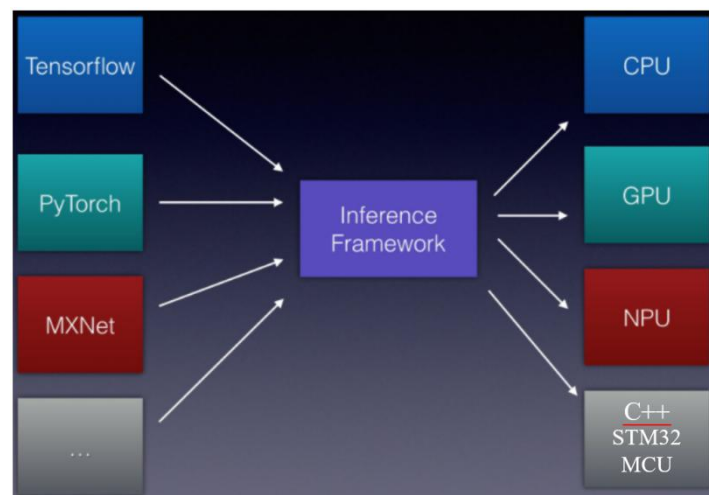


图2.5 推理框架原理示意

而在微控制器平台上，推理框架所要做的工作就是将训练好的模型转换成能被平台程序（对于STM32来说是C++）调用的优化代码和矩阵数组等文件。当前流行的工具包含TensorFlow-Lite Micro^[42]和专用于STM系列微控制器的ST X-CUBE-AI等。在学术界，MCUNet工作中Tiny Engine和Alessio Burrello等人提出的DORY也属于面向微控制器的推理框架。这些推理框架接收不同类型的训练模型，在库优化的层面针对不同硬件平台实现了平台相关的神经网络推理加速或更优内存调度等工作。当前已知最先进的面向微控制器的两种深度神经网络部署工具X-CUBE-AI和AutoTiler，是企业针对自己生产的微控制器提供的专用解决方案，不具备平台互通的特点。本课题所使用的便是X-CUBE-AI工具，通过其完成STM32微控制器深度神经网络模型的转换推理。

在完成对已训练模型的转换推理后，深度神经网络已经转换成平台IDE可加载的C++程序。深度神经网络部署的第三步则是将转换后的网络模型装载至对应

的微控制器应用中，包含例如摄像头、显示屏等外设调用的神经网络外的应用相关程序，完成最终的应用部署。本课题则借助FP_AI_Vision——一个由ST公司提供的视觉任务处理应用，完成基于压缩算子群的动态深度神经网络部署，并进一步在三个图像分类任务上进行了验证，这三种任务均来自于实际生活中的智能应用——智能餐品分类、智能垃圾分类以及智能交通标志识别，其将在2.4节进行更详细的阐述。

综上，深度神经网络于微控制器的整个部署流程如图2.6所示。

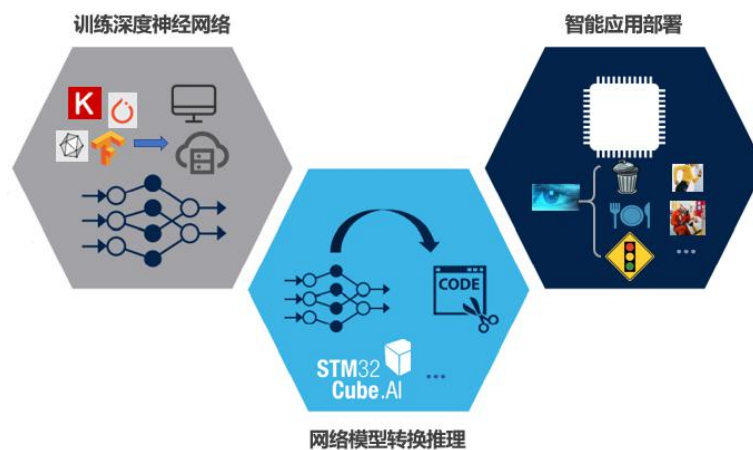


图2.6 微控制器上的深度神经网络部署流程

2.3.2 深度神经网络推理过程中的存储/内存管理

在资源条件中，持久性存储和内存的容量是微控制器相比于移动设备和传统CPU/GPU计算平台最为明显的差异之一。图2.7已对微控制器与智能手机和云服务器的存储/内存预算进行了对比。同时，存储/内存的容量是影响部署于微控制器上深度神经网络大小和尺寸的决定性因素；模型推理过程中的内存调度管理更是其中最关键的挑战之一。

（1）Flash和RAM

神经网络的推理涉及到两种类型的数据：动态数据（易失性数据，Volatile Data）和静态数据（非易失性数据，Non-volatile Data）。动态数据则指神经网络推理过程中所产生的的中间激活和进入到神经网络的输入缓存，被存放于易失性存储（内存），也即RAM（随机存取存储器）中；静态数据则指神经网络的权重和偏置，被存放于非易失性存储（或持久性存储），也即Flash（只读闪存）中。此外，在实际应用中，内存还需存放除神经网络之外的执行中数据，例如应用任务中的摄像头帧抓取，帧预处理等。

综上，微控制器上的深度神经网络及其推理过程必须满足内存和持久性存储两方面的资源限制。而由于微控制器本身极端受限的资源条件，在部署深度神经

网络之前，微控制器相比于GPU或移动设备等存储/内存资源更为宽裕的计算平台，需要更加精确的资源使用估算^[33]或测量，对于本课题模型需要提前装载多种不同压缩算子的情况更是如此。而深度模型在推理过程中的资源实际使用情况不仅与模型本身的结构特性和计算模式相关，还与硬件平台的底层支持、2.3.1中模型转换所使用的推理框架以及模型装载的实际应用等相关。这也进一步说明了，真实硬件指标的指导对于深度神经网络部署过程和此课题所涉及的动态自适应机制的必要性。

因此，下一部分将对微控制器（以本课题使用的STM32系列为例）上深度神经网络推理过程中的实际内存调度管理进行阐述。

（2）模型推理过程中的存储/内存管理

为了有效实现微控制器上基于压缩算子群的动态深度神经网络部署，本课题在模型设计层面具备一定的前提要求，受限于STM32系列微控制器本身的底层支持，遵循X-CUBE-AI的模型推理框架以及使用FP_AI_Vision任务应用。相关具体要求如下：

- ① 深度神经网络的设计不具备分支结构，不存在并行的计算模式。所有运算符在网络结构确定之后将以预定义的顺序执行，一次一个。
- ② 任何动态数据，包含摄像头的抓取帧（或从SD-Card卡读取的图片）、帧图像处理缓存（将帧或读取图片转换成神经网络的输入要求）、神经网络的图像输入缓存（NN_input_buffer）、神经网络的中间激活缓存（Activation buffers，包含了中间层的Input_buffer和Output_buffer），都仅存在于RAM中。这些数据的存放位置和调度管理由所使用的推理框架和平台底层的支持决定。在此次课题所使用的STM32微控制器上，其中的部分是可以根据实验需求进行设置的，相关具体配置详见2.3.3。
- ③ 由于STM32系列微控制器采用的依然是冯·诺依曼体系结构，任何静态数据，包含神经网络的权重和偏置以及代码的二进制文件都仅存放于Flash中。
- ④ 神经网络的中间激活缓存所处的RAM地址空间是一个随推理过程不断被分配、释放的复用空间，且其必须存在于一段连续的RAM地址空间中。此部分RAM所需的最大容量由两个连续层的最大内存需求决定（也遵循于X-CUBE-AI推理框架的内部优化等），即

$$RAM\ Size \approx Max(Activation\ Size) + Max(Layer\ Computing\ Needs) \quad (2-1)$$

综上，通过了解应用在运行过程中真实的存储/内存管理，整个流程Flash和RAM的调度管理情况如图2.7简要展示。

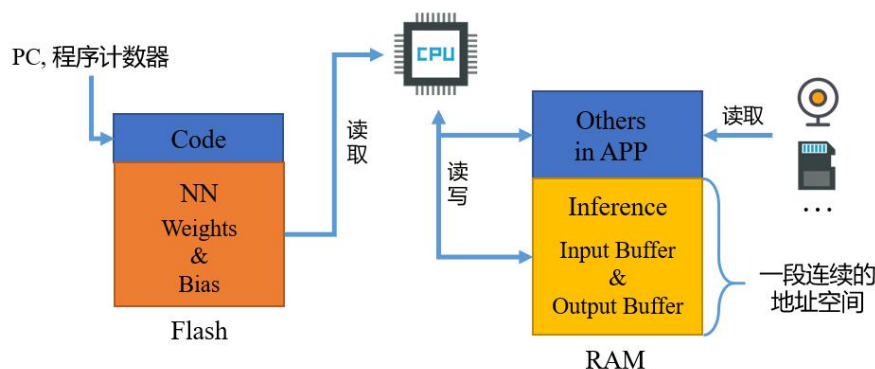


图2.7 STM32微控制器上的深度学习运行存储/内存示意

2.3.3 实验平台及模型推理相关配置——STM32H747

2.3.2对STM32系列微控制器的存储/内存管理进行了简要的介绍。本小节将对本课题所使用的STM32H747平台参数以及具体模型推理相关配置进行说明。

STM32H747属于STM32系列中的高性能微控制器。具体的，其搭载了ARM Cortex-M7和ARM Cortex-M4双核处理器，最高配频分别为480MHz和240MHz；拥有2MB的Flash和1MB的SRAM容量空间。为了开发过程的高效性，本课题使用了STM32H747I-DISCO开发板，其集成了STM32H747XIH6微控制器，具备外部SDRAM、外部Flash以及音频、SD卡、显示屏等外设接口，如图2.8所示。



图2.8 STM32H747I-DISCO开发板外观

深度学习实际部署情况中，上述参数中值得注意的细节有：

- ① RAM的1MB容量不是一个连续的地址空间。而前述提到神经网络的中间激活和输入必须放入一段连续的RAM地址内，所以实际模型中间激活和

输入缓存的最大限制是RAM中的AXI SRAM地址空间——512KB。

- ② 神经网络的权重和偏置应当小于且不接近于Flash的2MB上限，因为Flash中还存放了应用程序的二进制代码。
- ③ STM32H747I-DISCO开发板配置了256Mbit的外部SDRAM和2*512Mbit的外部Flash，用于分别对内部1MB SRAM和2MB Flash的补足。但是为了体现微控制器平台本身的极端资源限制对深度神经网络部署的影响，本课题将不使用上述外部Flash的资源补足。而在RAM方面，本课题将应用中与深度神经网络模型无关的摄像头获取帧、摄像头存帧、图像帧处理、和神经网络输入的缓存均放入至外部SDRAM，而只将由深度神经网络本身结构特性和运算模式决定的神经网络中间激活缓存和输出缓存放入内部SRAM中，具体内容和原因如下：

以一个采用VGA分辨率（640*480）摄像头取帧，和采用浮点运算/存储及224*224分辨率的彩色图像作为深度神经网络输入的应用任务为例，RAM中在此应用上需要装载的所有缓存类型、大小及流程如图2.9和图2.10所示。

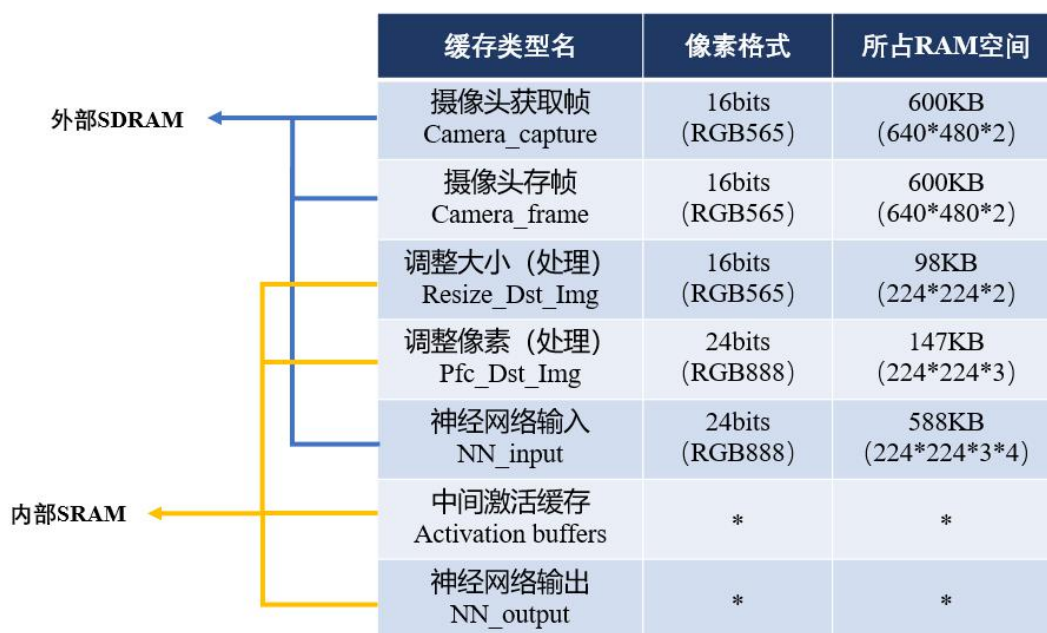


图2.9 本课题应用任务框架中的缓存类型和大小

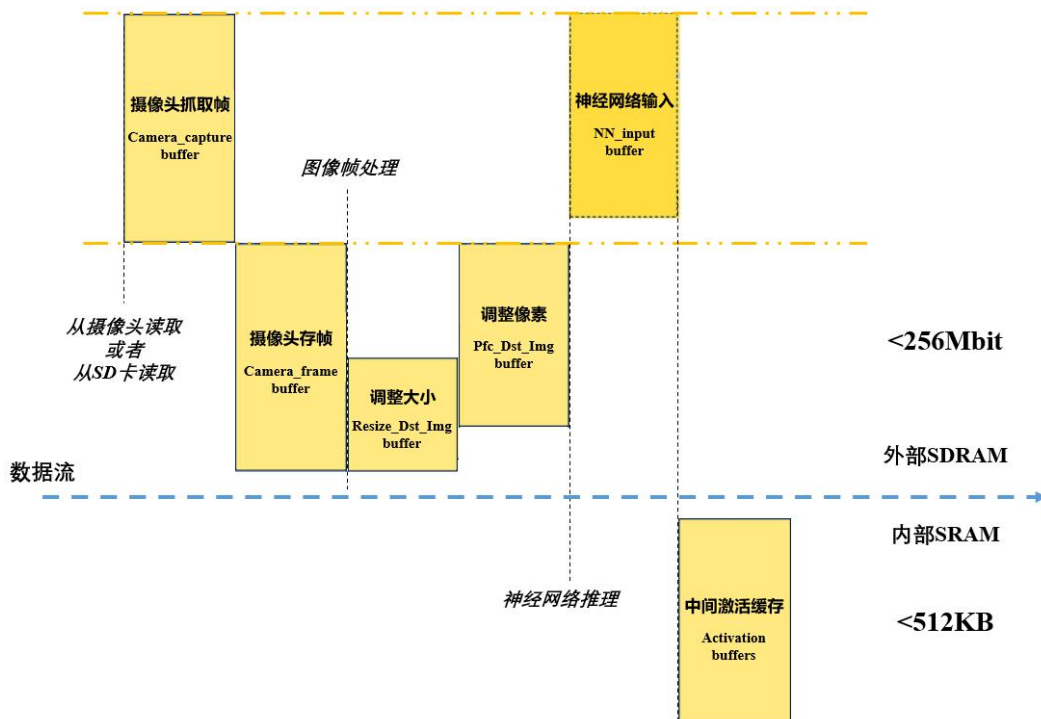


图2.10 本课题应用任务框架中的内存管理调度

图2.9中，摄像头获取帧也可以从SD-Card直接获取而得，只需要将图片的分辨率和像素格式对应设置即可。由图可知，在本课题使用的应用任务框架中，Camera_capture（600KB）、Camera_frame（600KB）、NN_input（588KB）所需内存空间均已超过了内部SRAM（512KB）的容量上限，因而只能被存放至外部SDRAM中。而对于图像帧处理缓存Resize_Dst_Img和Pfc_Dst_Img，虽其不超过内部SRAM的容量限制，也不会占用推理时的内存空间，但其具有至少占用147KB内部SRAM空间的要求，不利于探索深度神经网络本身与微控制器的资源约束和需求之间的关系，故放入外部SDRAM中进行管理。

综上，除了深度神经网络本身结构特性和运算模式决定的中间激活缓存被分配于内部SRAM中，应用中的其它缓存均放入至外部SDRAM。因此，上述的配置方案能够让应用中与模型无关的资源使用尽量不去影响深度神经网络的推理过程，使得该应用中内部SRAM和Flash等微控制器自带原生资源的使用情况仅由深度神经网络自身的模型结构和运算模式决定。

2.4 基于微控制器的三种实际智能图像分类场景

1.2节已经对基于微控制器的智能应用现状和前景进行了较为充分的阐述。本课题模型的实验验证，并没有使用传统图像分类任务常用的经典数据集，如Cifar-10、Cifar-100和ImageNet等，而是从生活中选取了三种真实的图像分类智

能应用场景——智能餐品分类、智能垃圾分类以及智能交通标志识别任务。此小节将对这三项图像分类智能应用场景和具体使用的数据集进行简要的介绍。

（1）餐品分类场景

食物是日常生活的重要组成部分。社交网络、专门的照片分享网站和移动应用程序中大量的美食照片清楚地说明了这一点。而餐品、菜肴的自动识别不仅可以帮助用户毫不费力地组织他们的日常照片，而且还可以使在线图片库的内容更易于访问。此外，移动食物摄影现在被用来帮助病人估计和跟踪他们的日常卡路里摄入量，摆脱了目前依靠营养专家^[43]的系统。同时，随着生活便利化和潮流导向，无人自助餐厅中对于餐品的自动分类和结账也成为了一种需求。

在数据集选取方面，本课题使用了斯坦福大学于 2014 年发布的Food-101数据集^[44]。其包含了101种餐品类别的图像数据集，共有 101,000 张图像。每个类别的测试图像和训练图像分别有 250 张和 750 张，其中训练图像未经数据清洗，但所有图像均已重新进行尺寸缩放，最大边长在 512 像素。在实际使用中，本课题只使用了其中18类餐品，共18000张图像。

（2）垃圾分类场景

随着经济社会发展和物质消费水平大幅提高，人们生活垃圾产生量迅速增长，环境隐患日益突出。2017年，国务院发布《生活垃圾分类制度实施方案的通知》，将垃圾的有效分类和处理正式提上国家日程。近年来，北京、上海等大城市起到表率作用，在垃圾分类工作上的成效显著。同时，繁琐的垃圾分类流程也为人们的生活增添了不便利性，而针对于垃圾分类的智能应用技术能够有效减轻人们的负担，对环境保护更为友好。

本课题具体使用了一个生活垃圾数据集，包含玻璃、纸、硬纸板、塑料、金属和一般垃圾6种类别，共2527张图片，且所有图片均是在日光或室内等真实场景条件下拍摄的。

（3）交通标志识别场景

伴随城市化进程的不断加快，车辆日益增多。而汽车数量的增加必将引起环境污染、交通拥堵等各类问题，仅采用交通管制或引导的传统方法很难达到问题解决的预期目标。此时，智能交通系统的出现更好地满足了当前交通系统的发展要求。其中最重要的一部分技术便是交通标志的自动识别，其成为车辆控制子系统研究的关键。同时，交通标志的智能识别分类也是无人车等未来驾驶场景必须攻克的技术要点。

本课题基于图像分类的交通标志识别具体使用了BelgiumTS交通数据集。其包含了7095张，共62类的图片，从不同角度拍摄和截取了真实场景下的交通标志。



图2.11 基于微控制器的三种实际智能图像分类场景

综合上述三种真实的图像分类智能应用的介绍，所使用的三项数据集具备不同的规模，且均取自于真实场景，能够从数据规模和真实应用两个层面证明本实验模型的鲁棒性。同时，基于微控制器价格低廉、功耗极低等优势，将这些任务部署于微控制器上，可以在公共领域和商业中进行广泛的部署和应用，极大程度地方便人们的生活。

第三章 基于压缩算子群的轻量化深度模型压缩方法

3.1 引言

随着神经网络在性能上的不断突破，模型本身的深度和宽度也逐渐扩展，运行时的时间复杂度和空间复杂度等指标也呈现出了爆炸式的增长。因而，深度神经网络的运行对计算平台的处理、存储能力提出了更高要求，加大了模型在资源受限的移动或嵌入式设备上的部署难度，特别以资源约束极端有限的微控制器为代表。这样的背景下，剪枝、量化等模型压缩和加速技术及轻量化的神经网络架构设计为上述挑战带来了可能的解决方案。本课题则选择微控制器这种成本能耗极低、适用范围极广，但在存储资源（运行内存和持久性存储）上相比于移动设备（如智能手机）还要小2~3个数量级的嵌入式终端作为实验平台，利用压缩算子和量化等一系列压缩方案完成深度神经网络的部署和推理，并通过三种真实的图像分类智能应用加以验证。

要实现微控制器上的神经网络成功部署和运行，与平台资源实际指标、骨干模型和压缩方案的选取和设计息息相关。其中本课题所使用的STM32H747微控制器参数已在第二章做了相关介绍。在骨干模型的选取上面，本课题选取了AlexNet^[2]，其作为ImageNet LSVRC-2010的冠军，在ImageNet数据集上具有37.5%的top-1错误率和17%的top-5错误率，超过了之前的工业水平，后续更是成为了神经网络最具代表的工作之一。除了其显著的地位，AlexNet本身的卷积层、全连接层、ReLU和Dropout等经典卷积神经网络设计，更是成为了后续模型设计的实现参考和众多压缩技术的验证载体。但在本实验所使用的数据集上，AlexNet所需存储是微控制器内部存储容量（2MB Flash）的116倍以上，不可能进行直接地部署和应用。

此章首先基于原始AlexNet挖掘深度神经网络的模型相关指标、真实平台开销与平台资源之间的关系和指导模型设计及压缩的相关机理，从而借助全局平均池化、模型结构同比例缩放等完成适配微控制器的轻量化变体AlexNet模型设计。同时，作为模型动态自适应的基础，此章将介绍基于压缩算子群的模型装载框架和所使用的模型量化压缩技术，使得轻量化变体AlexNet及其压缩算子群能够进一步部署和应用于微控制器平台上。最终本课题实现了微控制器上不同配置压缩算子装载模型平均百KB级的轻量部署，在餐品分类任务上实现平均70%的top-1精度，更在运行时延方面达成了有效的缩减。

3.2 模型结构和压缩方案设计流程

本节将细致介绍基于压缩算子群和AlexNet的轻量化模型结构和压缩方案设计流程。首先，此部分将对原始AlexNet的结构特性和相关指标（模型相关指标与真实平台开销）等进行测量和说明，并结合微控制器平台的资源等完成相关分析，探索适配微控制器的轻量化模型设计和压缩方案实施的指导机理。

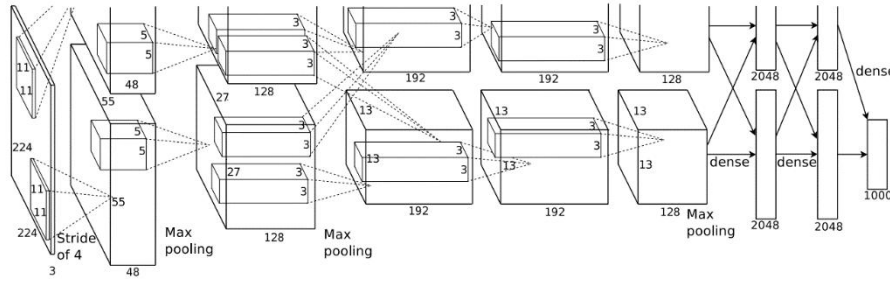


图3.1 AlexNet原始结构示意图

图3.1为AlexNet的原始结构示意图，从中我们可以看到：

- ① 原始的AlexNet是一个8层的卷积神经网络——其中5层为卷积层、3层为全连接层。在卷积层中，AlexNet采取了96-256-384-384-256的卷积核设置，具有“中间宽，头尾窄”的结构特性。
- ② 采用了“卷积-LRN-ReLU”的计算模式设计。其中LRN由AlexNet第一个提出，指的是局部响应归一化，即Local Response Normalization，对局部神经元的活动增加竞争机制，使得其中响应比较大的值变得相对更大，并抑制其它反馈较小的神经元，增强了模型的泛化能力。而ReLU^[45]

（Rectified Linear Unit）作为线性整流函数，是一种人工神经网络中常用的激活函数(activation function)，其计算式如公式3-1所示。相比于sigmoid和tanh，ReLU 计算模式简单，能够显著加快训练速度，且被证明更加有效和符合生物上的合理性。

$$f(x) = \max(0, x) \quad (3-1)$$

- ③ 在前两个卷积层的激活函数后，增加了最大池化层（MaxPooling）的操作，将中间输出特征尺寸进一步缩减。使用224*224的彩色图片作为输入时，AlexNet在餐品分类数据集上的各层信息如图3.2所示。其中，卷积层使用“Same_padding”的卷积模式；最大池化层都采用核为(3, 3)，步长为2的池化模式；全连接层采用了50%比例的Drouout；输出层采用softmax；卷积核只使用核宽和核高表示，省略输入输出维度。

编号	计算模式	卷积设置	输出特征	特征尺寸	参数量	计算量 (MACC)
Conv1	Convolution→LRN→ReLU→MaxPooling	核: (11, 11) 步长: 4	(27, 27, 96)	69984	35328	110816448
Conv2	Convolution→LRN→ReLU→MaxPooling	核: (3, 3) 步长: 1	(13, 13, 256)	43264	615680	448847104
Conv3	Convolution	核: (3, 3) 步长: 1	(13, 13, 384)	64896	885120	149520768
Conv4	Convolution	核: (3, 3) 步长: 1	(13, 13, 384)	64896	1327488	224280960
Conv5	Convolution→MaxPooling	核: (3, 3) 步长: 1	(6, 6, 256)	9216	884992	149603584
FC1	Fully Connected	*	4096	4096	37752832	37752832
FC2	Fully Connected	*	4096	4096	16781312	16781312
FC_preds	Fully Connected	*	18	18	73746	73746
Total	*	*	*	*	58356498	1137676754

图3.2 原始AlexNet各层模型相关指标

由上图数据可得，原始AlexNet在模型相关指标上呈现如下规律：

- ① 特征尺寸：在卷积层，由于卷积运算、最大池化运算的特性，神经网络中间层输出特征的宽度和高度随着模型推理过程趋于减小，从而也导致特征尺寸的降低。
- ② 参数量：卷积层的参数量取决于当前层卷积核的高度、宽度以及输入、输出特征的维度。因而在AlexNet卷积层中，前四层的参数量随着模型加深而不断增大。此外，由卷积层和全连接层的参数量对比（红色颜色深度）可发现，全连接层的参数量几乎占到了整个模型的93.45%，是AlexNet中参数量占比最大的计算结构。
- ③ 计算量：卷积层的计算量取决于当前层卷积核的高度、宽度、个数及运算步长和输入特征尺寸。由图可知，和模型参数量所表现的规律截然相反的是：原始AlexNet的计算量集中于卷积层，占到了整个模型的96.96%，且前两层卷积层的计算量之和大于后三层。

上述三项模型相关指标在一定程度上反映了AlexNet各层对于不同资源（计算、存储）的开销。但仅以此作为AlexNet压缩的指导指标，进行微控制器平台的模型部署还不具备完善的说服力。因为特征尺寸、参数量、计算量与模型在微控制器（或其它平台）上的真实资源消耗如内存使用情况、时延、能耗等虽具有一定的关系，但并不能直接进行等效替代。例如，Yang等^[46]已提到仅仅降低模型的参数量和计算量并非一定能使模型运行的开销或时延降低。所以在对AlexNet于微控制器平台的部署进行压缩方案设计之前，我们需要清楚模型相关指标和模型真实平台开销的关系，并从中提炼指导模型设计和压缩的机理。

由第二章相关技术综述已知，神经网络中间层的特征尺寸决定了对应层的输出激活缓存大小，而中间层的缓存激活大小又决定了该层对于RAM容量的空间需求。在X-CUBE-AI推理框架下，AlexNet所需要的最大RAM占用时刻为执行第三层卷积运算前——需要同时存放第二层输出激活缓存，并为第三层输出激活缓存预留空间。浮点型网络中，此时RAM需求为422.5KB $((43264+64896)*4/1024)$ ，如图3.3所示。此外，相比RAM使用情况更加简单的是，模型的参数量（主要由权重和偏置组成）直接决定了平台持久性存储的容量需求——原始的浮点型AlexNet需要约233.42MB的Flash空间。计算量与模型时延和能耗的关系相对于参数量和计算量则更为复杂，现有研究也主要集中于定性分析或通过另设预测模型来对原始网络进行预测^[47]。但Liberis等人^[33]也通过实验结果指出由于微控制器通常缺乏随时改变性能的特性，如一般以固定频率在单核处理器上运行，所以模型计算量和微控制器上该模型的运行时延具备非常高的拟合度（ $R^2=0.975$ ），一定情况下可以用计算量作为模型时延的代替指标。

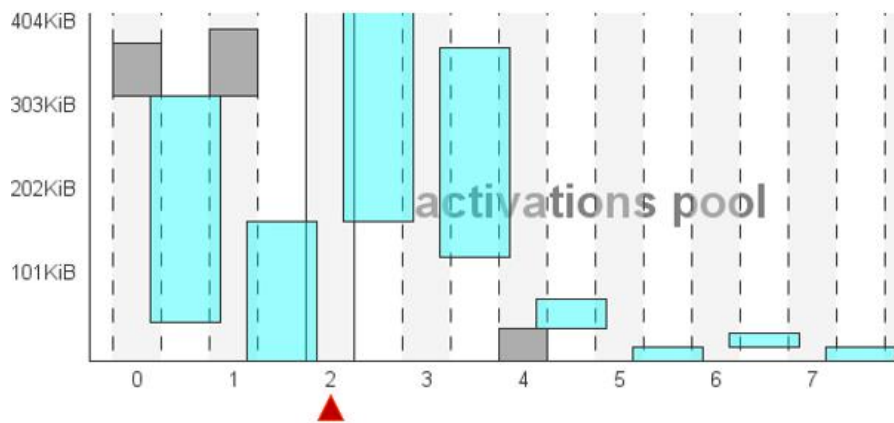


图3.3 AlexNet中间激活缓存（或RAM）的占用情况

综上，从原始AlexNet各层模型相关指标及其与微控制器平台真实资源开销的关系分析可以得到：原始AlexNet模型对微控制器平台的RAM需求(422.5KB)小于STM32H747微控制器的RAM容量上限（512KB），但是其对于Flash的容量需求（233.45MB）已经超过微控制器本身资源条件（2MB）的116倍以上。因此，要实现第一步微控制器平台上AlexNet变体模型的成功部署，最重要的压缩对象应为模型的参数量（或模型尺寸）。本课题以上述探索作为后续模型设计和压缩的指导机理，完成微控制器上的基于压缩算子群和AlexNet的轻量化变体模型结构和压缩方案的确定。整个流程如图3.4所示。



图3.4 模型结构和压缩方案设计流程

3.2.1 轻量化变体模型设计

由本节前述探索可知，模型参数量成为了AlexNet部署于微控制器上最大的条件制约。基于此，本小节将介绍采用全局平均池化层、模型结构同比例缩放等策略或思想对原始模型结构进行初步的压缩，首先进行适配微控制器平台的变体AlexNet模型设计。

（1） GAP

GAP是一种池化操作，其能将 (n, n, dim) 维度的输入特征转换为 $(1, 1, \text{dim})$ 的输出特征，本质是将 dim 维的 $n \times n$ 特征分别进行平均求值，因而名为全局池化。

GAP层于CNN的第一次成功应用是在Lin等人提出的NIN^[47]模型中，被用于替代空间复杂度极大的全连接层。在其之前，传统的CNN卷积运算一般出现在网络的低层。对于分类问题，最后的特征图往往与全连接层相连，并再通过softmax逻辑回归分类层进行整个网络的输出。这种情况下，我们可以把卷积层看做是特征提取器，然后再利用卷积运算得到的特征通过全连接层对输入图像进行分类。然而由前述分析可得，在AlexNet和VGGNet这样业界认可的模型中，全连接层的参数量基本约占到了整个模型的90%。冗余的参数量除了给模型存储带来巨大的负担以外，往往还容易使得模型出现过拟合的现象，导致网络的泛化能力不尽人意。Hinton等^[48]提出Dropout的方法，虽然一定程度上提高了网络的泛化能力，但并未减少模型的参数量。

GAP则利用全局平均池化的方法，替代了传统CNN中的全连接层。与传统的全连接层不同（见图3.5），在低层通过卷积运算获得特征图后，GAP对每个特征图进行全局平均池化，这样每张特征图就会转换为仅仅一个数值。其最大的特点是，GAP这样的结构甚至直接省去了原本全连接层所包含的所有参数，大大减小了网络的尺寸，并被验证能够一定程度避免过拟合的现象。但另一方面，由于GAP本身的特性，使用GAP层作为卷积层与输出之间桥梁的网络，需要将GAP层的前一层卷积输出维度调整为输出分类个数（例如在1000分类的任务中，最后一个卷积层的输出维度应设置为1000）。但在本课题中，变体AlexNet需要适应三种不同类型的图像分类任务。为了避免不同的卷积层设置为模型的效果带来的影响，我们将三种不同分类任务中所使用的AlexNet变体模型的低层卷积层设为完全一致，在GAP层之后再新增一个 $1 \times 1 \times \text{classifier_dim}$ 的卷积层（`classifier_dim`指该分类任务下的输出类别数），对整个网络的输出进行适配。

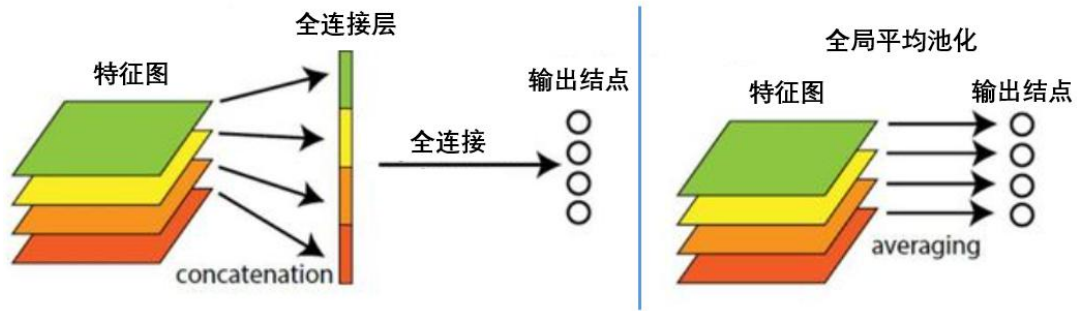


图3.5 全连接层操作（左）与全局平均池化操作（右）

通过利用GAP运算，AlexNet的结构从原来的8层转化为7层（5个卷积层+1个GAP层+1个适配用卷积层）。整体模型图将于后续给出。

（2）模型结构同比例缩放

使用GAP替代全连接运算后，我们已经使得原始AlexNet约90%的参数量空间被节省。但仅此，改变后的模型尺寸依然超过了微控制器Flash容量的10倍以上。在此之上，本课题在变体AlexNet上进一步采取了结构同比例缩放的策略和思想，具体的：我们保留了AlexNet“中间宽，头尾窄”的拓扑特性，将原始AlexNet采取的“96-256-384-384-256”的卷积核设置，进行了1/4的同比例缩放，即将卷积核数分别设置为24、64、96、96和64，但其余所有卷积核配置（如步长、大小等）保持不变。其目的是通过同比例缩放，在尽量保留原始AlexNet本身结构特性的前提下，初步设计出能够满足于微控制器资源限制的变体AlexNet模型。

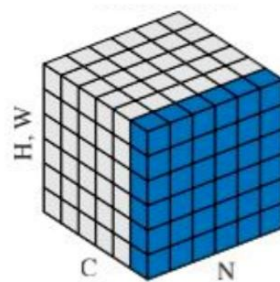


图3.6 批规范化（BN）示意

除此以外，相比较于原始的AlexNet模型，本课题还将局部响应归一化层（LRN）替换为批规范化层^[49]（Batch Normalization, BN）。BN于LRN三年后的2015年提出，如图3.6所示。在训练过程中的一批图像样本包含了 $N \times C \times H \times W$ 四个维度，其中N为样本数，样本通道数为C，高为H，宽为W。BN的具体操作是保留通道维度C，将一批次图像中的每一个样本的通道特征（N、H、W）进行平均，

得到每个通道的均值，然后再让每个像素（pixel）剪去该均值并除以方差，最终实现规范化的结果，见公式(3-2)。归纳原论文中对BN的描述：其从数据分布特征的角度出发，意图解决神经网络前向传播中的Internal Covariate Shift（ICS）问题——每一层的参数更新会导致上层的输入数据分布发生变化，使得高层需要不断地适应低层网络的参数更新，从而导致了模型训练收敛速度降低，以及往往需要非常谨慎地设定学习率、初始化权重和细致的参数更新策略等。BN最主要的作用就是通过批次均值和方差来调整每层特征的数据分布，加速神经网络收敛，并使模型训练更加稳定，从而降低对人工适应调整网络超参数的依赖。

$$x_{out} = \gamma \left(\frac{x - \mu_c(x)}{\sigma_c(x)} \right) + \beta \quad (3-2)$$

其中，均值和方差分别为：

$$\mu_c(x) = \frac{1}{NHW} \sum_{n=1}^N \sum_{h=1}^H \sum_{w=1}^W x_{nchw} \quad (3-3)$$

$$\sigma_c(x) = \sqrt{\frac{1}{NHW} \sum_{n=1}^N \sum_{h=1}^H \sum_{w=1}^W (x_{nchw} - \mu_c(x))^2 + \epsilon} \quad (3-4)$$

综上，本课题通过GAP和模型结构同比例缩放策略首先设计出一种轻量化变体AlexNet模型。通过上述轻量化模型设计方案，本课题生成的变体AlexNet在模型参数量、计算量等模型相关指标上进行了有效缩减。更重要的是，在X-CUBE-AI推理框架下，轻量化变体AlexNet相比原始AlexNet在RAM、Flash容量需求等真实平台开销方面大幅降低，能够成功适配于微控制器平台，且分类精度与FD-MobileNet^[50]（一种变体MobileNet模型）几乎具有同等水平。相关实验细节（包括其于三种图像分类任务的实际表现）将于3.3进行更详细地介绍。

本课题最终基于压缩算子群的模型自适应框架设计需要在变体AlexNet模型上，对多种压缩算子进行实现和装载，同时还需将本体模型与多种压缩算子一同部署于微控制器平台上，对Flash容量限制要求更为苛刻。故本节接下来的部分还将对基于压缩算子群的模型装载框架设计，以及使用量化策略进一步压缩轻量化变体模型的相关机制进行阐述。

3.2.2 基于压缩算子群的模型装载框架

在3.2.1小节，本课题从理论上已经完成了能够成功部署于微控制器平台的轻量化变体AlexNet模型设计。其过程中已经提到，仅与模型相关的指标（如参数量、计算量、特征尺寸等）并不能完全作为模型在平台上部署的指导指标。任何深度神经网络是否能够真正运行于对应平台不仅取决于模型本身的结构特性和

运算模式，更与推理框架的选择、计算平台本身的资源限制和底层/硬件的支持等有关。故真实硬件指标（如内存使用峰值、Flash容量占用、时延、能耗等）指导的深度神经网络部署更具实际意义。但同时，仅仅如此模型还不具备实现本课题中随着资源情境或需求进行自适应调优的能力，缺乏能使模型结构动态化的机制和框架设计。

在此小节，本课题提出了基于压缩算子群的模型装载框架设计，利用不同的压缩算子在3.2.1生成的轻量化变体AlexNet上进行实现和装载，进一步生成多种不同的压缩算子变体模型。其中，压缩算子群的设计则借鉴了近些年轻量化网络设计中最为突出的块构建思想，利用具备不同结构和计算模式的“块”，实现对本体模型的改造。由这些压缩算子派生而来的不同变体模型具备不同的性能和平台资源开销，在模型的分类精度、内存占用情况以及时延等方面具有不同的表现特征。以此构建起的模型搜索空间，是本课题实现情境自适应的基础。情境自适应框架将在第四章进行更为详细地介绍。

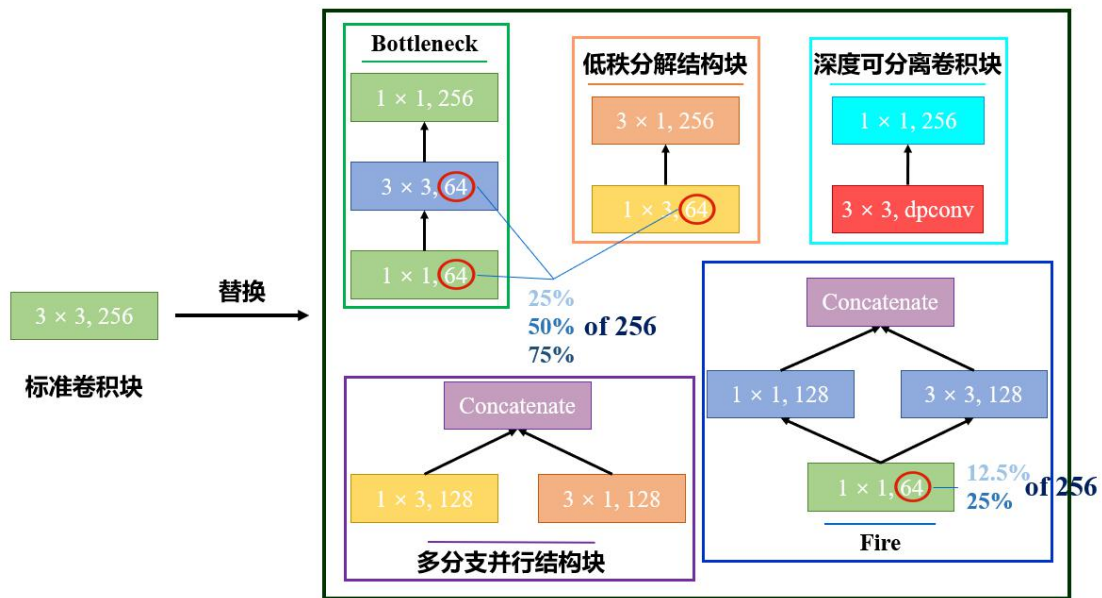


图3.7 本课题所使用的压缩算子结构块及相关配置

在压缩算子的选取方面，本课题借助了近些年在轻量化网络结构设计中被研究学者实验验证的高效运算“结构块”。这些结构块被用于替换AlexNet中的传统卷积块模式，不同程度地改变了本体模型的宽度、深度和连接性等结构特性与运算模式，从而改变了其于计算平台上的真实性能和开销。接下来，此小节将对本课题所选取的具体压缩算子进行简要介绍，并阐述其对应的压缩机理。本课题所有压缩算子结构块见图3.7。

（1） Bottleneck结构块

Bottleneck(沙漏型结构)作为早期轻量化网络模型中最为出名的构建块思想,在Inception^[51]、ResNet^[22]等网络中均有体现。其将1层传统卷积操作替换成了3层卷积,同时实现了模块轻量化的目标。以传统的 $3*3*256$ 的卷积操作为例,Bottleneck即是将该卷积按顺序替换成 $1*1*num$ 、 $3*3*num$ 、 $1*1*256$ 的卷积运算,其中num可设置为任意小于256(输出维度)的正整数,两个 $1*1$ 核大小的卷积运算则分别用于降低和升高特征维度。通过这样的结构,模型的深度被加深,但参数量和计算量却同时降低了。在本课题中,num的选取分别使用了输出维度的25%、50%和75%的取值。

（2） 低秩分解结构块

低秩分解结构块(例如Svd-Based^[52])则利用了一种低秩近似的技术,将传统卷积操作进行不对称二分量分解、三分量分解或四分量分解。本课题所使用的是一种二分量分解方法,如将传统 $3*3*256$ 的卷积运算顺序分解成 $1*3*num$ 和 $3*1*256$ 的两层卷积,加大模型深度的同时也降低了块的参数量和计算量,且num同样选取了输出维度25%、50%、75%的超参数配置。

（3） 多分支并行结构块

多分支并行结构块起源于Inception系列模型^[51]。与(1)和(2)的结构块不同,其认为通过加深深度但是减小中间输出维度的方式可能会造成信息损失。为了避免这样的情况,多分支并行结构从模型宽度层面出发在某一层采用了多组卷积运算,最后通过合并并将多组卷积运算的结果进行整合。本课题则采用了一种两组卷积的并行结构——将传统 $3*3*256$ 的卷积运算替换成两组 $1*3*128$ 和 $3*1*128$ 的卷积,并在最后将两组卷积的输出进行合并(Concatenation)。通过这样的方式,结构块的参数量和计算量也达到了减小的目的。

（4） Fire结构块

Fire压缩算子诞生于SqueezeNet^[23]——于2016年发布的最早期的轻量化网络模型之一。作为此篇工作的核心创新之一,“Fire结构块”利用一种

“Squeeze-Expand”的思想,本质类似(1) Bottleneck结构块和(3)多分支并行结构快的结合体。以传统 $3*3*256$ 的卷积运算为例,Fire结构块首先通过Squeeze操作——使用 $1*1*num$ 的卷积对输入特征进行降维(num一般取值小于输出维度),再使用Expand操作——通过 $1*1*e_1$ 与 $3*3*e_2$ 两种卷积对Squeeze操作后的输出特征再进行运算并最后对两种卷积的输出进行合并,即整个Fire块的输出维度等于 $e_1 + e_2$ 。本课题采用了两种Fire结构块的配置,即将num设置为整个结构块预定输出维度的1/8和1/4(上例中分别为32和64),同时 e_1 与 e_2 设置为相同大小(上例中为128)。从3.3的实验结果可看出,Fire结构块相比前三种压缩算子更进一步地压缩了模型的参数量和计算量。

（5）深度可分离卷积块

深度可分离卷积块由Google的MobileNet^[24]首次提出。类似Fire模块，深度可分离卷积也采用了两段式的卷积运算模式——深度卷积（Depthwise Convolution）和逐点卷积（Pointwise Convolution），见图3.8。与前述四种方法都不相同的是，其中深度卷积的操作并非传统的卷积运算——深度卷积只针对于单个通道（输入特征的一个维度）进行卷积，而不像标准CNN对所有输入通道进行卷积。因此，深度卷积可以理解为卷积核维度只能为1的卷积操作，且卷积核数量只能等于输入特征的维度。逐点卷积则是 1×1 * 整个块输出维度的传统卷积，作用是对深度卷积操作后的特征维度进行调整（升降维度），类似Bottleneck。深度可分离卷积的命名正是来自于其能够将卷积对特征深度、宽度的操作分离开来。相比于具有同样输入和输出特征的传统卷积，深度可分离卷积大大降低了卷积块的参数量和计算量。以输入为 $7 \times 7 \times 3$ ，输出为 $5 \times 5 \times 128$ 的某相邻输出特征为例，传统的核大小为 3×3 、步长为1的卷积操作的计算量为 $128 \times (5 \times 5) \times (3 \times 3 \times 3) = 86400$ ，参数量为 $128 \times (3 \times 3 \times 3) = 3456$ ；而深度可分离卷积的计算量仅为 $3 \times (5 \times 5) \times (3 \times 3 \times 1) + 128 \times (5 \times 5) \times (1 \times 1 \times 3) = 10275$ ，参数量仅为 $3 \times (3 \times 3 \times 1) + 128 \times (1 \times 1 \times 3) = 411$ 。

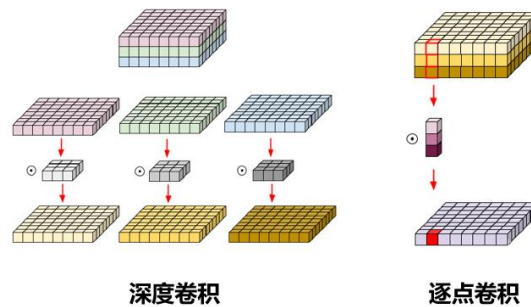


图3.8 深度可分离卷积示意

综上，对于这些压缩算子更为详细的指标及实验对比将在3.3实验结果部分进行更详细地介绍。至此，我们已经构建起拥有不同配置的压缩算子群，以它们为结构块组成的深度神经网络具备不同的深度、宽度、连接性等结构特性与运算模式，从而在微控制器平台上具备不同的真实性能和开销表现。在本课题中，我们希望通过将这些不同的压缩算子结构块实现并装载于3.2.1生成的轻量化变体 AlexNet上，并以此作为模型自适应机制的实现基础。因此，此部分还将对基于压缩算子群的模型装载框架及设计原因进行简要地介绍，但不会展开叙述自适应相关机制（此内容将于第四章进行详细地介绍）。

要使得平台上的智能应用能够根据不同的响应条件（如资源情境、输入样本等）进行动态调整，一个最为质朴的想法^[40]为：在平台上部署多个具有不同水平的性能和开销间权衡的网络模型，在适配响应时利用一定的映射机制选取当前最

优的网络模型并部署。但由前文探索可知，微控制器及其它移动或嵌入式设备在资源容量上最为受限，这种质朴的模型部署思想实现无疑会为平台造成极大的负担，甚至完全不可行。相比于这样的方案，本课题的基于压缩算子群的模型装载框架以3.2.1中适配微控制器的轻量化变体AlexNet作为本体模型，通过装载不同类型和不同配置的压缩算子（Bottleneck结构块、低秩分解结构块、多分支并行结构块、Fire结构块以及深度可分离卷积块）能够显著应对上述挑战。

在实际使用中，本课题采用了权重共享和结构块替换的框架设计——具体的，在轻量化变体AlexNet的基础上，将前两个卷积层的权重进行共享并且保持传统的卷积块结构（不采取任何压缩算子的装载），后三个卷积层进行压缩算子结构块的替换和训练，完成具备不同性能和开销的模型自适应空间构建，如图3.9。值得注意的是，在每一层进行装载时，所有压缩算子结构块的输入、输出维度保持一致，以保证可替换性。最终本体模型和不同配置的压缩算子将按顺序一同存放到Flash的一段连续空间内，这种框架设计的具体原因和优势有：

- ① 相比于多种不同模型之间进行完全替换，基于压缩算子结构块的替换方案更加节省微控制器平台上最为珍贵的内存空间。
- ② 轻量化变体AlexNet的后三层卷积占用模型近90%的参数量（见3.3实验结果展示），对其进行压缩算子装载有利于整体模型尺寸的缩减，进而能够释放更多微控制器平台的Flash容量空间。且不同压缩算子派生的变体模型共享了低层（前两层）结构，保留了图像浅层的特征提取模式，通过控制变量的思想更易于探索和比较不同压缩算子的性能和开销。3.3实验结果展示部分也证实了这种模型装载设计的有效性。
- ③ 采用了轻量化变体AlexNet作为本体模型，且不同压缩算子相比传统卷积块也更为轻量，因而对于其在微控制器平台的部署相对友好。此外，不同表现的压缩算子也从模型层面为后续情境自适应框架设计提供了多样的搜索空间。

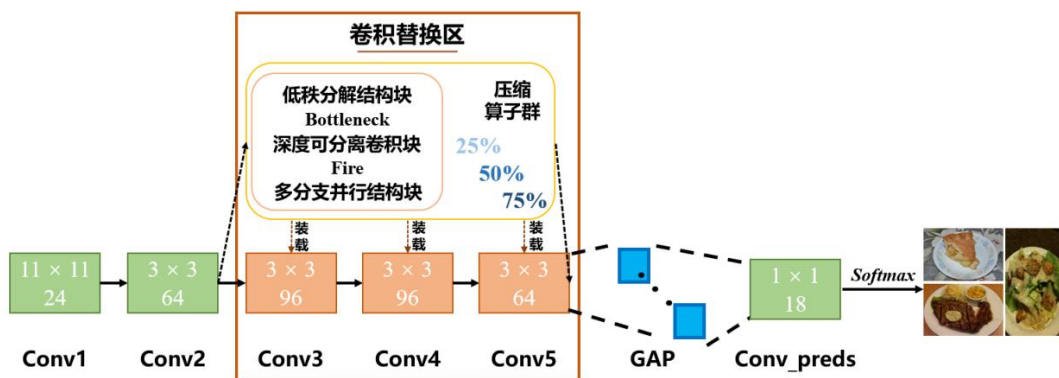


图3.9 本课题使用的基于压缩算子群的模型装载框架设计

3.2.3 模型量化压缩策略

在3.2.1和3.2.2中，本课题已经完成了轻量化变体AlexNet模型设计和基于压缩算子群的模型装载框架设计。这些轻量化设计和压缩方案选择已经一定程度地完成了适配微控制器平台资源的神经网络部署工作。本课题在此基础上再次使用了一种在资源受限平台上颇为有效的压缩技术——模型量化，更进一步地对本体模型及其压缩算子于微控制器平台的资源开销进行了优化。本小节则将对本课题所使用的模型量化技术进行介绍。

2.2.1相关技术综述已经对模型量化技术进行了简要的说明：模型量化是一种通过降低网络参数、中间激活特征、甚至输入输出表示所需的比特数来减小原始模型尺寸，对模型推理和训练进行加速的模型压缩方法。其可以将32-bit（浮点型）的数值表示减小到16-bit、8-bit（整型）、甚至1-bit（二值化网络），但同时对模型分类精度只造成较小的影响。本课题采用了一种8-bit训练后量化方法——在完成浮点型变体AlexNet及压缩算子群的训练之后，对网络参数和输入、输出及中间激活特征都进行8-bit整型的量化映射。参考Jacob等人^[53]的整型推理算法，其量化映射可表示为如下公式：

$$r = (q - \text{zero_point}) \times \text{scale} \quad (3-5)$$

$$q = \frac{r}{\text{scale}} + \text{zero_point} \quad (3-6)$$

式(3-5)表示了量化后数值 q 至浮点值 r 的映射模式，式(3-6)反之。其中 scale 为量化后的最小刻度， zero_point 表示0浮点值对应的量化后数值。其中 scale 和 zero_point 的求值公式如下：

$$\text{scale} = \frac{R_{\max} - R_{\min}}{Q_{\max} - Q_{\min}} \quad (3-7)$$

$$\text{zero_point} = Q_{\max} - R_{\max} \quad (3-8)$$

其中 $R_{\max}$ 表示实际最大浮点值， $R_{\min}$ 表示实际最小浮点值， $Q_{\max}$ 表示最大量化后数值， $Q_{\min}$ 表示最小量化后数值。对8-bit整型量化来说，量化后的数值范围为 $[-128, 127]$ ，因此 $Q_{\max} = 127$ ， $Q_{\min} = -128$ 。

实验结果表明，本课题所采用的8-bit（整型）训练后量化技术有效地减小了模型的参数量和计算量，从而对模型尺寸进行了4倍的缩减，有效提升了模型的运行速度，且精度损失小于2%，更多实验结果将在3.3进行详细地展示。

3.3 实验结果展示

本章实验结果展示部分，将使用三种图像分类任务（主要以智能餐品分类为展示样例），从轻量化变体AlexNet模型验证、基于多配置压缩算子群的模型装载效果、模型量化及平台部署三个方面进行介绍。

3.3.1 轻量化变体模型验证

在3.2.1中，本课题结合对微控制器平台资源限制和原始AlexNet结构特性与相关指标的分析，得到模型尺寸（取决于模型参数量）是深度神经网络于微控制器平台部署的最大制约。因而，本课题融合全局平均池化层、模型结构同比例缩放思想及批规范化处理等，完成了一种轻量化变体AlexNet的模型设计。其于三种智能图像分类任务的实际精度表现如表3-1所示。

表3-1 轻量化变体AlexNet与两种实验对比

智能图像分类任务	轻量化变体AlexNet	原始AlexNet	MobileNet变体
餐品分类	73.80%	57.28%	67.63%
垃圾分类	74.87%	51.53%	74.05%
交通标志识别	96.23%	71.10%	97.07%

表3-1中，MobileNet变体则使用了Zheng等人提出的Fast Downsampling MobileNet^[50]（FD-MobileNet），其采用快速下采样策略在原始MobileNet的基础上对网络进行进一步加速，具备很高的轻量化特性，模型参数量与轻量化变体AlexNet保持相当。从表中可以分析得到：在三项图像分类任务中，轻量化变体AlexNet和FD-MobileNet几乎具备同等精度水平，但是原始AlexNet的实际表现却远远差于其它两种模型。通过原始AlexNet的训练收敛过程分析，本课题猜测：原始AlexNet的全连接层设计和繁重的参数量（61M）增大了其对于数据集的拟合难度（本课题使用SGD对原始AlexNet的训练均在300轮以上，使用Adam优化对其它两种轻量化模型的训练均在120轮以内），同时本课题所使用的真实图像分类数据集在数据量上不具备ImageNet和Cifar等经典验证数据集的规模水平（特别是垃圾分类和交通标志识别任务，见2.4）。这些原因都可能导致了原始AlexNet在三项任务中的表现较差。

通过上述精度对比实验说明，轻量化变体AlexNet在本课题的三个真实智能图像分类场景下的可行性，其与FD-MobileNet几乎具备同等精度水平，也证明了本课题的轻量化设计的有效性。而在模型相关指标方面，以餐品分类任务所使用的变体AlexNet为例，其各层模型相关指标如图3.10所示。

编号	计算模式	卷积设置	输出特征	特征尺寸	参数量	计算量 (MACC)
Conv1	Convolution→BN→ReLU→MaxPooling	核: (11, 11) 步长: 4	(27, 27, 24)	17496	8808	27553584
Conv2	Convolution→BN→ReLU→MaxPooling	核: (3, 3) 步长: 1	(13, 13, 64)	10816	14080	10221760
Conv3	Convolution→BN	核: (3, 3) 步长: 1	(13, 13, 96)	16224	55680	9361344
Conv4	Convolution→BN	核: (3, 3) 步长: 1	(13, 13, 96)	16224	83328	14033856
Conv5	Convolution→BN→MaxPooling	核: (3, 3) 步长: 1	(6, 6, 64)	2304	55552	9376640
GAP	Global Average Pooling	*	(1, 1, 64)	64	*	2304
Conv_preds	Convolution	核: (1, 1) 步长: 1	(1, 1, 18)	18	1170	1440
Total	*	*	*	*	218618	70550928

图3.10 轻量化变体AlexNet各层模型相关指标

从图3.10中可以看到，通过此小节的轻量化模型设计方案，本课题生成的变体AlexNet相比原始AlexNet（见图3.2）在参数量上实现了近似266倍的缩减，计算量约为原始的1/16。在X-CUBE-AI推理框架下，变体AlexNet的RAM容量需求为105.7KB，约为原始的1/4，小于微控制器512KB的RAM容量空间，见图3.11。在浮点型网络下，变体AlexNet的权重和偏置等参数所占用的Flash容量预估为853.98KB，也已小于微控制器2MB的Flash容量空间。至此，轻量化变体AlexNet理论上已经可以成功部署于STM32H747微控制器之上。

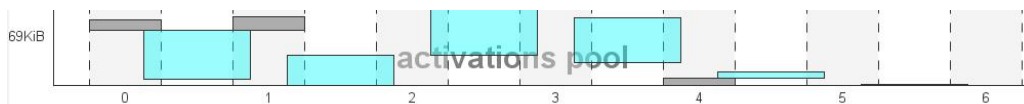


图3.11 变体AlexNet中间激活缓存（或RAM）的占用情况

至此，通过全局平均池化、模型结构同比例缩放等设计，本课题生成的轻量化变体AlexNet，不仅在三项真实图像分类场景下实现与FD-MobileNet同等精度水平，更在模型相关指标和真实平台开销上进行了有效轻量化，达到适配微控制器资源限制的标准。后续基于压缩算子群的模型装载效果和模型量化效果均将以此轻量化变体AlexNet作为Baseline进行比较分析。

3.3.2 基于多配置压缩算子群的模型装载效果

完成轻量化变体AlexNet设计之后，在3.2.2中本课题进而提出一种基于压缩算子群的模型装载框架，作为后续情境自适应的基础——在变体AlexNet的基础上，通过低层权重共享，高层卷积压缩算子替换的方式，生成了不同配置压缩算

子装载的新变体模型群。本小节则将对本课题所使用的压缩算子群配置，及不同压缩算子的实际表现进行详细地展示（由于篇幅限制，只使用智能餐品分类任务作为示例）。

如3.2.2所述，本课题使用了五种类型的压缩算子：**Bottleneck**结构块、低秩分解结构块、多分支并行结构块、**Fire**结构块以及深度可分离卷积块。其中由于结构块不同的超参数选择，**Bottleneck**结构块和低秩分解结构块根据中间卷积核的维度可分为块输出维度25%、50%、75%三种配置，**Fire**结构块根据“Squeeze”阶段的降维程度也可分为块输出维度12.5%和25%的两种配置。故加上本体模型（不进行任何压缩算子装载的轻量化变体**AlexNet**），本课题基于压缩算子群的可选择模型空间维度为11。

除本体模型外，10种压缩算子装载后的模型训练均采用了如下训练配置：使用Adam优化算法，在前80轮进行batchsize为32、学习率为0.001的训练，在后70轮进行batchsize为16、学习率为0.0002的训练，整体训练轮次数为150。

首先，图3-12展示了这11种配置模型的精度对比。

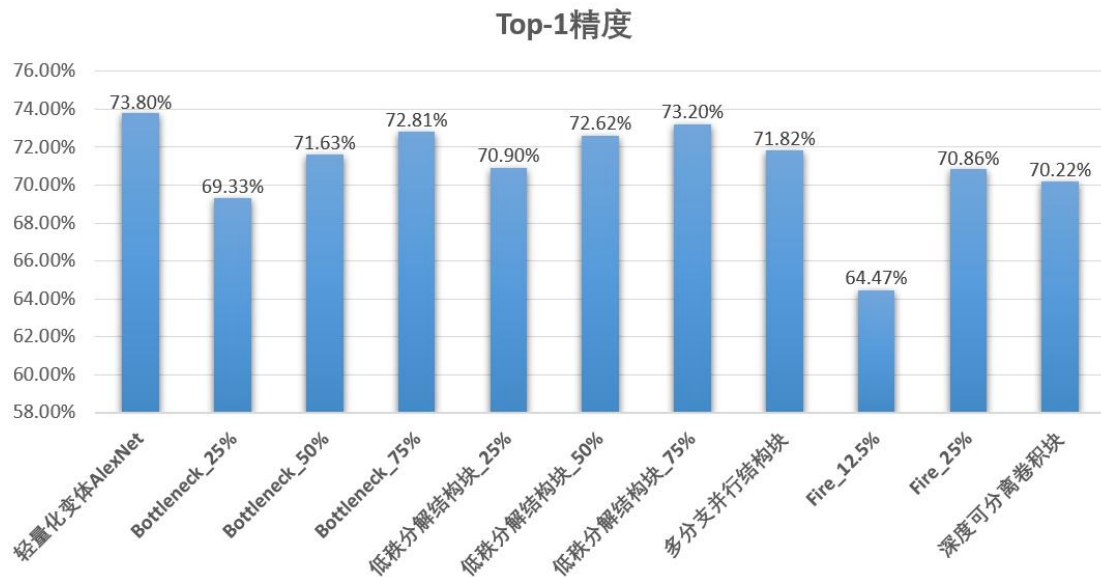


图3.12 各配置模型Top-1精度对比

由图3.12可分析得到，相比于本体模型（轻量化变体**AlexNet**），不同配置压缩算子装载的模型在精度水平均有一定的损失。且由**Bottleneck**、低秩分解结构块和**Fire**不同压缩比配置的算子装载效果可看出，压缩程度越高的配置模型精度损失相对越大。但压缩程度越高也意味着模型轻量化水平越高，图3-13至图3-17展示和对比了本体模型及其于不同配置压缩算子装载下的参数量、计算量的模型相关指标以及在微控制器上的所需RAM容量、所需Flash容量、单样例运行时延真实平台开销。

（1）模型相关指标

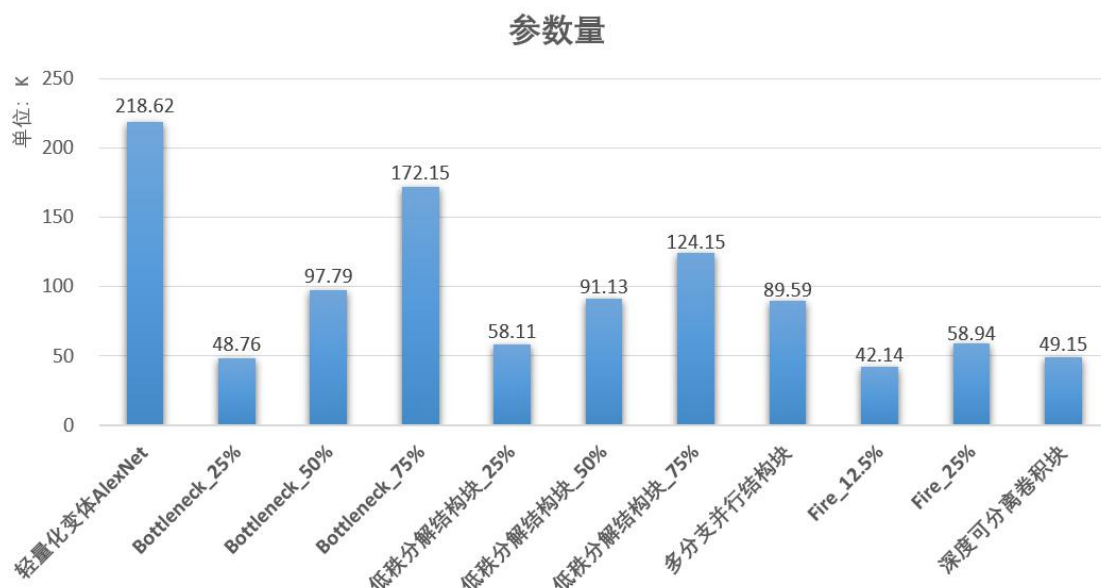


图3.13 各配置模型参数量对比

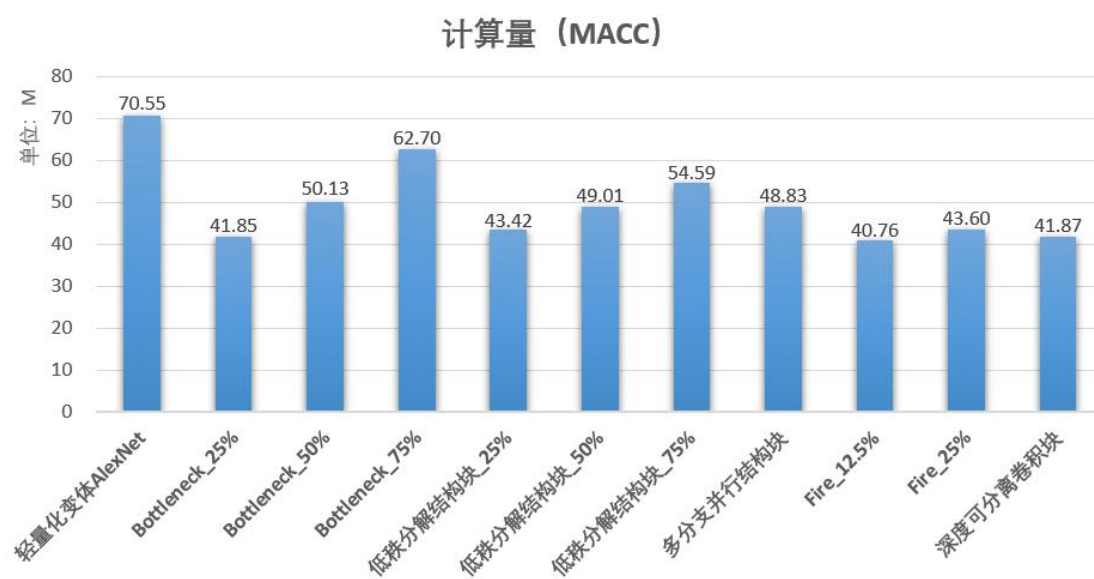


图3.14 各配置模型计算量对比

（2）真实平台开销

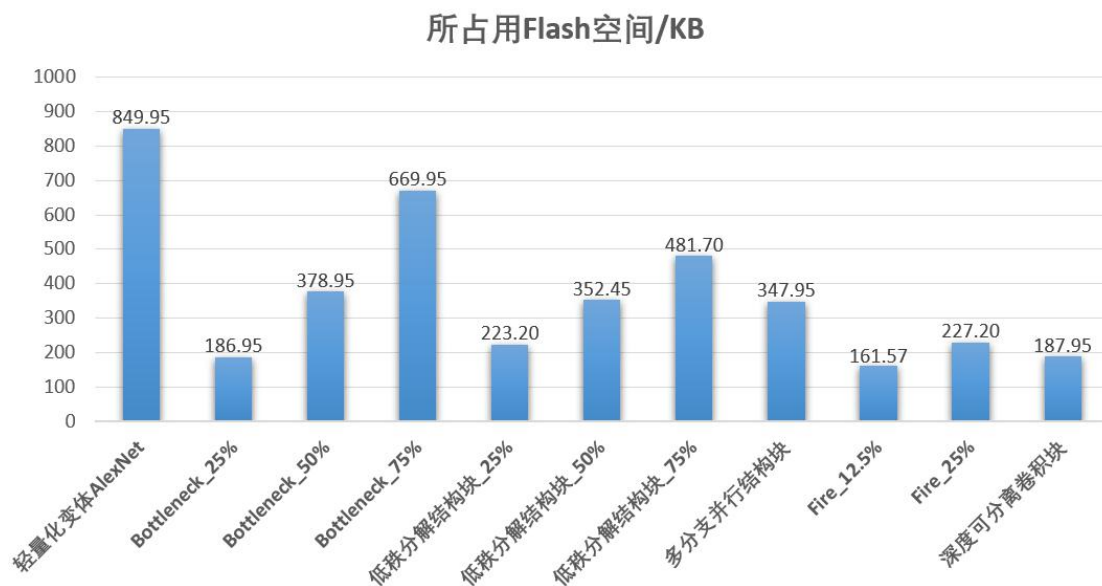


图3.15 各配置模型所占用Flash空间对比

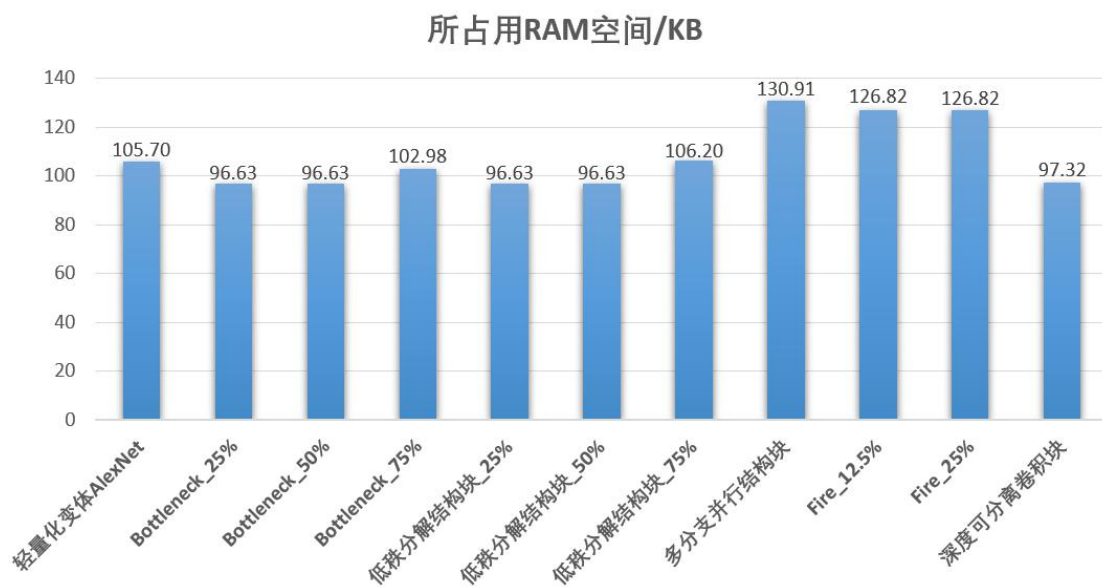


图3.16 各配置模型所占用RAM空间对比

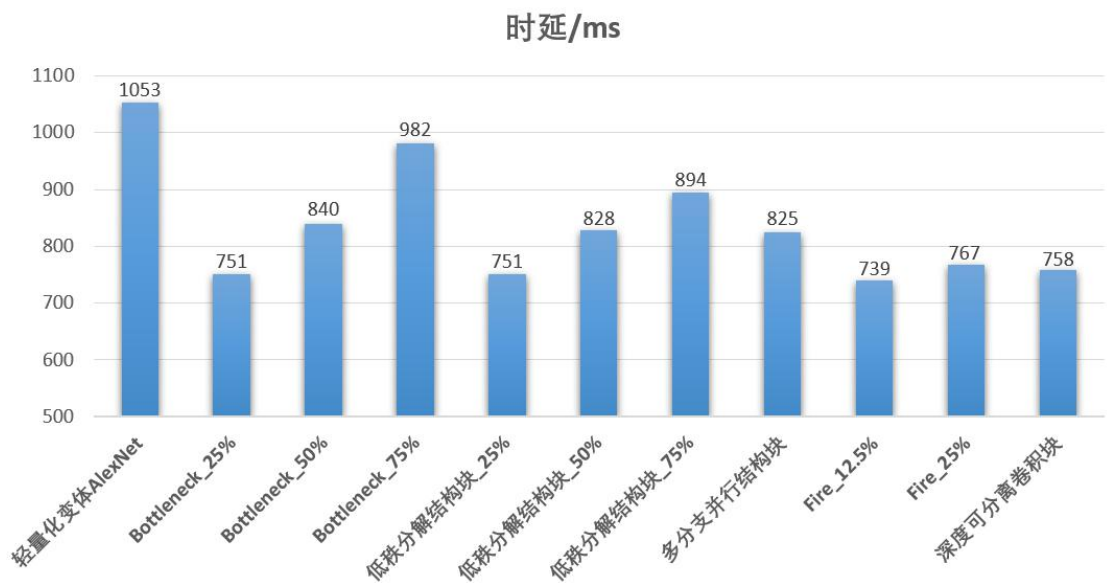


图3.17 各配置模型单样例运行时延对比

依据上述实验结果，本课题归纳：

①除了参数量和所占用Flash空间存在明显的相关关系，在微控制器平台上，不同配置压缩算子装载的模型的分类精度、单样例运行时延和参数量、计算量同样存在高度的规律一致性和相关性。其Pearson相关系数（值越接近于1，说明两序列越呈正线性相关）如表3-2所示。其中，时延与计算量和参数量的相关系数高达近0.998，很好地验证了“计算量可作为微控制器上卷积网络模型时延的代替指标”这一机理^[33]。“精度-参数量”、“时延-计算量”的线性拟合关系分别如图3.18和图3.19所示。

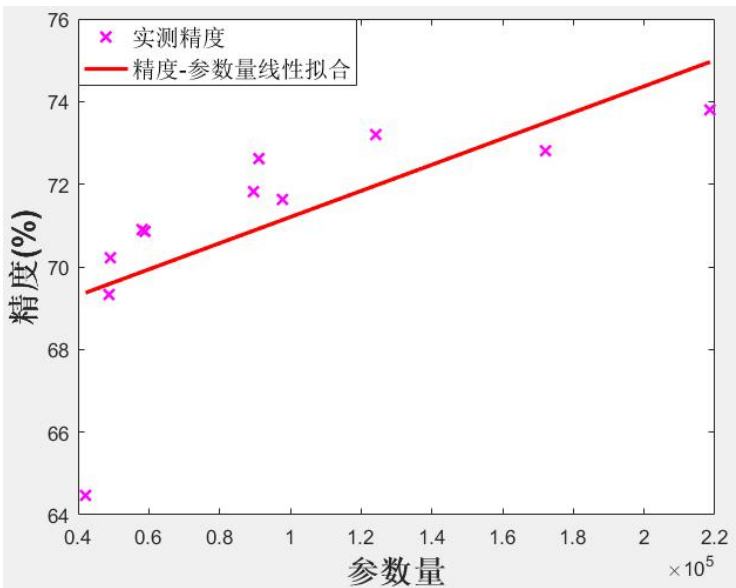


图3.18 压缩算子群派生模型精度和参数量的线性拟合

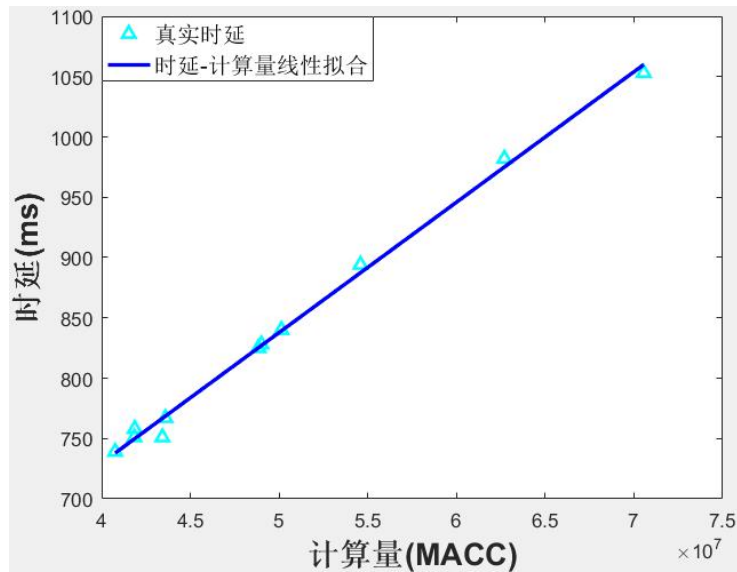


图3.19 压缩算子群派生模型时延和计算量的线性拟合

表3-2 精度、时延与参数量、计算量的相关性

指标序列对	皮尔逊（Pearson）相关系数
精度与参数量	0.69685
精度与计算量	0.69644
时延与参数量	0.99788
时延与计算量	0.99784

②通过图3.16所占用RAM空间的真实平台开销对比可以发现，模型实际占用的RAM大小与模型本身的参数量和计算量并无直接关系。实际上，2.3.2已经提到，模型推理过程中的中间激活特征大小才是真正影响其于微控制器上RAM占用大小的决定因素。本课题中的多分支并行结构快和Fire结构块均具有多分支的卷积块设计，在模型推理过程中产生的中间激活特征尺寸相比于其它单分支压缩算子配置的模型要大，因而RAM的实际占用空间也相对较大。

③本课题基于不同配置压缩算子的装载设计为模型带来了参数量、计算量等模型相关指标的改变，更不同程度地影响了模型分类精度和在微控制器上的真实平台开销，不仅进一步地压缩了轻量化变体AlexNet，还形成了一个多样的具备不同实际表现的压缩算子群模型空间，作为后续情境自适应的基础。同时，从图3.12至图3.17可以看出，“Fire结构块”、“深度可分离卷积块”、“低秩分解结构块_25%”和“Bottleneck_25%”等轻量化程度越高的压缩算子装载方案，造成的精度损失也越大。

下一小节则将对在上述实验基础上进行的模型量化技术和最终基于压缩算子群的模型平台部署进行介绍。

3.3.3 模型量化及平台部署

在3.3.2实验部分，本课题已经完成了基于压缩算子群的模型装载设计，对本体模型和不同配置压缩算子装载后的模型进行了指标对比，在进一步压缩模型的基础上形成了多样的变体模型空间。在实际部署前，本课题使用了一种8-bit整型训练后量化技术，对模型再次进行压缩，量化前后的分类精度对比如图3.20所示。



图3.20 量化前后模型分类精度对比

由图3.21可知，8-bit训练后量化技术给模型带来了不超过2%的精度损失。但是由于将模型的输入、输出，中间激活特征以及权重参数从32位浮点型替换至8位整型表示，模型所占用Flash空间以及所占用RAM空间在原压缩算子模型群的基础之上再次形成了4倍的缩减。此外，上述量化方案还加速了模型的推理速度，平均实现了微控制器上3倍单样例运行时延的降低，如图3.21所示。

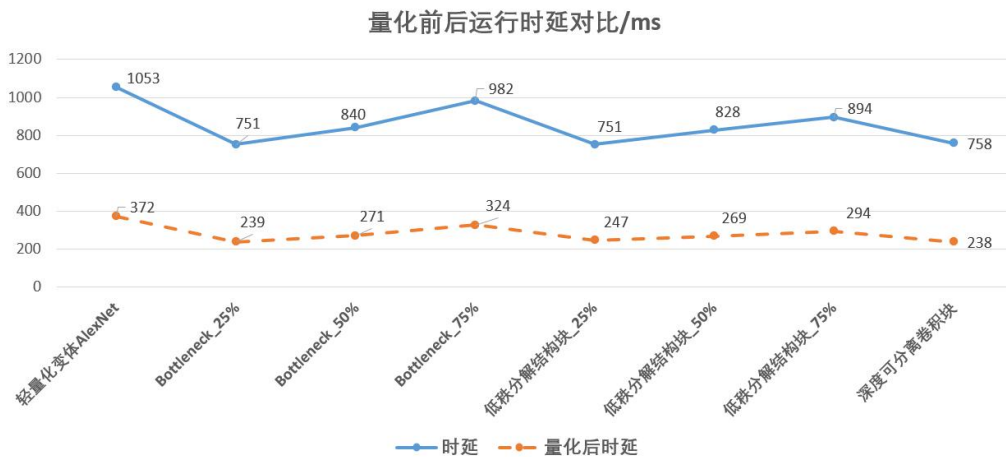


图3.21 量化前后单样例运行时延对比

至此，本章整个基于压缩算子群的轻量化模型设计和压缩流程已经完成。通过上述流程，量化后压缩算子装载模型相比本体模型（量化前的轻量化变体 AlexNet）存在平均约3%的精度损失，实现了最低50KB以内级别的模型生成，部分模型效果对比如表3-3所示。

表3-3 本课题部分模型效果对比

模型	精度	所占用Flash空间（KB）	时延（ms）
轻量化变体AlexNet	73.80%	849.95	1053
Bottleneck_50%	70.12%	97.79	271
低秩分解结构块_50%	71.80%	91.13	269
深度可分离卷积块	68.74%	49.15	238

通过量化，最后本课题采用了如图3.22所示的微控制器实际部署方案，将本体模型及其压缩算子群成功部署于STM32H747微控制器平台上。其中，本体模型放于Flash空间后，压缩算子群则按照顺序同样存放至Flash中。在模型推理环节，除了结构共享区（前两层）的使用固定外，应用可以按照地址选择后三层的装载方案（本体模型或不同配置的压缩算子群）。另值得注意的是，为了保证精度，压缩算子群和本体模型的训练过程中，“Conv_preds”层也进行了变化。因此，后三层选择装载方案的同时也需要装载其对应的Conv_preds层（量化后的Conv_preds层大小仅约1KB，因而不会过分影响整体的装载效果）。

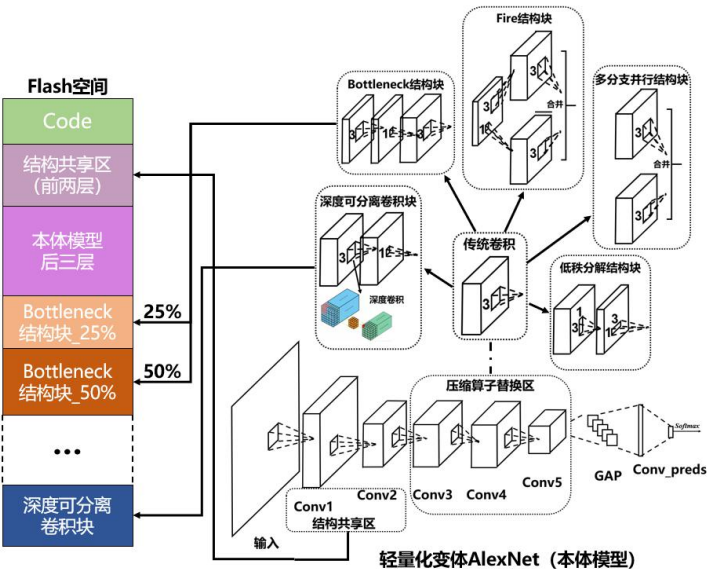


图3.22 本体模型及其压缩算子群的实际部署方案

第四章 面向微控制器的软硬协同情境自适应框架

4.1 引言

静态深度模型在平台和智能应用中的成功部署并不能完全解决应用在运行过程中造成的“资源过载或浪费”的自适应能力差问题。特别是对于资源受限、低能耗需求高的设备（例如微控制器）来说，如何使模型在不仅满足平台资源限制的情况下，更尽可能地适应于平台资源情境或应用实际需求，实现精度和开销的巧妙权衡以及对情境的最优适配更成为越来越重要的待解决问题。在第三章，本课题已经完成了面向微控制器的基于压缩算子群和AlexNet的轻量化模型结构及压缩方案设计。由3.2可知，本课题通过全局平均池化、压缩算子装载、模型量化等流程设计实现了STM32H747微控制器上深度神经网络的成功部署，且通过基于压缩算子群的装载框架完成了模型的动态机制基础。在本章，本课题将利用第三章生成的模型进一步完成模型自适应整体框架的搭建。

在基于变体AlexNet和压缩算子群装载的动态机制确定后，如何从这些不同表现（如分类精度、资源开销）的压缩算子中选取当前情境最优的压缩方案或模型调整策略将成为本章讨论的核心之一。此外，本课题进一步提出了一种面向微控制器的软硬协同情境自适应框架。软硬协同中的“软”指在应用模型层面，利用前述基于压缩算子群的模型装载设计作为情境自适应的基础；“硬”则指在硬件平台层面，利用基于微控制器的电压频率调整技术增效模型自适应策略搜索空间。这种软硬协同的框架设计，具有相比于单独模型或平台层面设计更多样的策略空间，能够更多维地实现情境的最优适配，且两个层面的设计流程相对独立、互不影响，具有很高的可拓展性。

本章将对整个软硬协同情境自适应框架进行说明和建模，对整体框架设计、建模指标选择等进行完备地阐述，最后通过实例验证整体框架的可行性。

4.2 软硬协同情境自适应框架建模

在对本课题软硬协同自适应框架进行介绍之前，我们需要对模型自适应的对象——情境，进行定义和说明。情境可分为平台资源情境和用户情境：平台资源情境指的是平台的电量、存储等平台自身的资源约束；用户情境指的是用户或智能应用在运行时对模型的时延、能耗甚至精度等提出的自定义需求。因此，本课题所谈及的软硬协同情境自适应框架，目的是能够在模型和底层两个层面对模型所处智能应用的精度效果和资源开销进行自适应调优，使其完成对情境的最优适

配，见图4.1。所以，本课题自适应的本质就是一个根据情境在调优空间中进行策略选择和部署的搜索问题。本节则将对上述搜索问题进行建模，对其涵盖的内容进行逐一阐述。

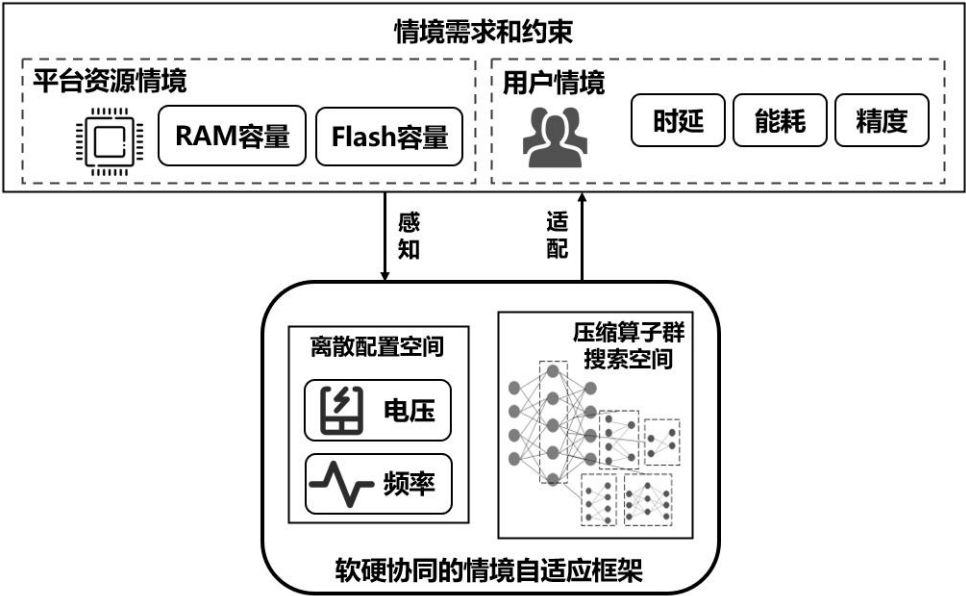


图4.1 软硬协同的情境自适应框架

对搜索问题的建模可以分为三个部分：搜索指标——进行自适应的启动和依据指标；搜索空间——定义了可以搜索的智能应用部署方案，即解的空间；搜索策略——从情境到方案确定的映射机制和评估方法。本课题中，搜索指标即是前述提到的情境变化，搜索空间则由基于压缩算子群的模型动态框架和微控制器平台的电压频率调整策略协同设计决定，搜索策略则转换为情境约束下的离散多目标优化。

4.2.1 情境指标选择及其约束下的多目标优化搜索策略

如3.2节所述，深度神经网络是否能够成功部署于资源受限平台设备上不能只关注于模型参数量、计算量等仅与模型本身结构和运算模式相关的参数指标，必须综合考虑推理框架、平台底层/硬件支持等因素，使用模型的真实平台开销指导其部署过程。而模型情境自适应更是如此，结合本节对情境的定义，本课题关注的微控制器情境指标与对应模型真实平台开销如下：

- (1) RAM与运行内存使用峰值（Peak Memory Usage, PMU）
- 2.3.3已经对本课题实际使用的，深度神经网络在微控制器平台运行过程中的内存调度管理进行了较为详细地说明。从中可知，微控制器的内部SRAM需要随时分配/释放推理过程产生的中间激活缓存，这些正存储于RAM中的缓存张量被

称为工作集（Working Set）。而工作集的最大尺寸即代表了该模型在运行时的内存使用峰值（PMU），因而有：

$$PMU(M, a) \leq RAM(t) \quad (4-1)$$

其中 M 指的是当前使用的神经网络本体模型， a 表示模型选取的压缩算子装载方案， $RAM(t)$ 表示当前时间 t 下预估的剩余内存容量。值得一提的是，实际情况由于其它应用、外设或内核程序的占用， RAM 的剩余容量往往是很难测出的，且模型的 PMU 一般应当小于且不接近于当前智能应用的实际使用内存。但本课题，使用了微控制器上单个的智能图像分类应用和2.3.3特殊的内存管理调度方式，所以使得模型运行的 PMU 尽可能地能够代表该智能应用当前真实的内存占用情况。

（2）Flash与模型尺寸（Model Size, MS）

Flash作为只读内存，在微控制器中起到的是持久性存储的作用。在本课题中，Flash需要装载智能应用的二进制程序（包含了深度神经网络拓扑结构及其它图像处理的所有可运行代码）以及本体模型和压缩算子群权重及偏置。所以Flash限制了模型层面自适应框架设计的多样化空间，但相比于 RAM 与模型 PMU 的关系更为简单——在运行时，Flash的容量空间和内容不会发生改变，因而有：

$$Code + Storage(M, A) \leq Flash \quad (4-2)$$

A 表示的是本课题需要提前装载的压缩算子群。在实际情况中，由于Code由模型所处的应用框架决定，往往不会发生太大的变化，故本课题使用Flash容量上限减去Code所需空间之后的容量剩余 $Flash^*$ 作为限制本体模型和压缩算子群设计空间尺寸的情境指标，如下：

$$MS(M, A) \leq Flash^* \quad (4-3)$$

（3）运行时延（Latency）和能耗（Energy Efficiency, EE）

模型的部署和运行必须满足上述 RAM 和Flash等微控制器平台自身的资源情境指标约束。与其不同，模型运行时延和应用能耗作为用户或智能应用的自定义情境需求，并非应用成功部署的“硬性”约束，但也同等重要，例如不同应用对于响应实时性和应用续航性的高需求和常见于物联网终端设备中的低能耗运行模式等。本课题采用软硬协同的自适应框架设计，形成了具备不同运行时延和能耗表现的方案搜索空间。 s 代表当前微控制器平台的电压、频率调整策略方案：

$$Latency(M, a, s) < needed\ Latency \quad (4-4)$$

$$EE(M, a, s) < needed\ EE \quad (4-5)$$

本课题软硬协同策略下的模型运行时延可由实际测量获取，并存放于静态表格中。能耗的测量则相对困难，其可以采用Roofline模型^[54]中提到的计算强度 I 作为模型能耗的代替指标，但这样做的缺点也十分明显——使用仅与模型相关的代替指标与应用模型的真实能耗具有一定的差距，且本课题基于微控制器的电压、频率调整技术将不会造成该指标的变化。

至此，本课题对选取的情境指标及其与模型真实平台开销之间的关系进行了介绍和分析。上述指标中，Flash容量空间在模型部署步骤前完成对本体模型和压缩算子群的资源情境约束；RAM则在模型运行时对其实际的内存使用峰值进行资源限制；运行时延和能耗则作为用户情境对应用模型的运行性能提出需求和约束。完成情境指标的确定后，即可进行对情境自适应框架的搜索策略设计。在本课题中，其可以表示为软硬协同策略空间下带情境约束的多目标优化问题：

$$a^* = \operatorname{argmin}_{a \in A, s \in S} L(M, a, s) \quad (4-6)$$

$$\begin{aligned} L(M, a, s) = & L\{1 - \operatorname{Acc}(M, a), \\ & MS(M, a), PMU(M, a), \\ & Latency(M, a, s), EE(M, a, s)\} \\ s.t. & PMU(M, a) \leq RAM(t), MS(M, A) \leq Flash^* \end{aligned} \quad (4-7)$$

公式中， a^* 表示的是当前情境下的软硬协同策略选择（包含了压缩算子和电压、频率两个层面）， L 表示的是确定策略的带约束多目标函数。据此公式，由于用户情境的软特性，本课题将时延、能耗两项情境指标与模型精度一起作为优化问题的目标，而非条件约束。至此，情境指标的选择及其至方案空间的映射机制或搜索策略设计已经完成。接下来，本节则将对软硬协同的自适应策略搜索空间进行细致说明。

4.2.2 软硬协同的自适应策略搜索空间设计

本课题中，情境自适应的搜索空间由两部分组成：一部分是基于压缩算子群的变体模型空间 A ，另一部分则为基于微控制器平台的电压、频率调整空间 S 。在情境自适应的过程中，模型所装载的智能应用利用4.2.1中的搜索策略，在由 A 和 S 所组成的联合搜索空间 A^* （ $a^* = (a, s) \in A^* = (A, S)$ ）中进行策略选择。由于压缩算子群的框架设计已在3.2.2进行了完备的叙述，本小节将侧重于介绍本课题中微控制器上的电压频率调整技术的策略空间设计及相关原理。

动态电压频率调整技术（DVFS）是一种广泛应用于便携式设备（如智能手机、MP3、笔记本电脑等）的节能技术。其根据芯片所运行的应用程序对计算能力的不同需要，动态调整芯片的电压和运行频率（对同一芯片来说，频率越高，对应所需电压也越高），从而达到节能的目的^[55]。其理论依据如下：

$$P = \beta CV^2 F \quad (4-8)$$

$$E = P * t = \beta CV^2 F * t \quad (4-9)$$

上式中， β 属于缩放因子， C 代表设备电容， V 代表电压， F 为运行频率。从中可以看出，单独降低频率 F 不一定能够节省能量（对于特定任务， $F * t$ 为常量， F 的降低会导致运行时延 t 的增大），所以同时降低频率和电压才可以真正地降低能量的消耗。至此可以看出，DVFS中的相关机理能够很好地被借鉴并使用于本

课题的情境自适应框架——通过不同的电压、频率配置，使得智能应用的运行性能和开销发生变化，从而组成微控制器层面的策略搜索空间，实现与模型层面的协同设计。具体的，本课题所使用的微控制器电压、频率配置及相关原理如下：

（1）频率配置

在STM32系列微控制器上配备了HSI、HSE、LSI、LSE、PLL五种时钟源，不同时钟源具备不同的输出频率。在实际中，往往使用HSE和PLL作为系统时钟SYSclk的输入，这是因为HSE和PLL具备更高的频率上限，系统时钟在分频后能为设备其它模块的时钟（例如Fclk、Hclk、Pclk等）提供更大的频率调整空间。STM32时钟系统如图4.2所示：

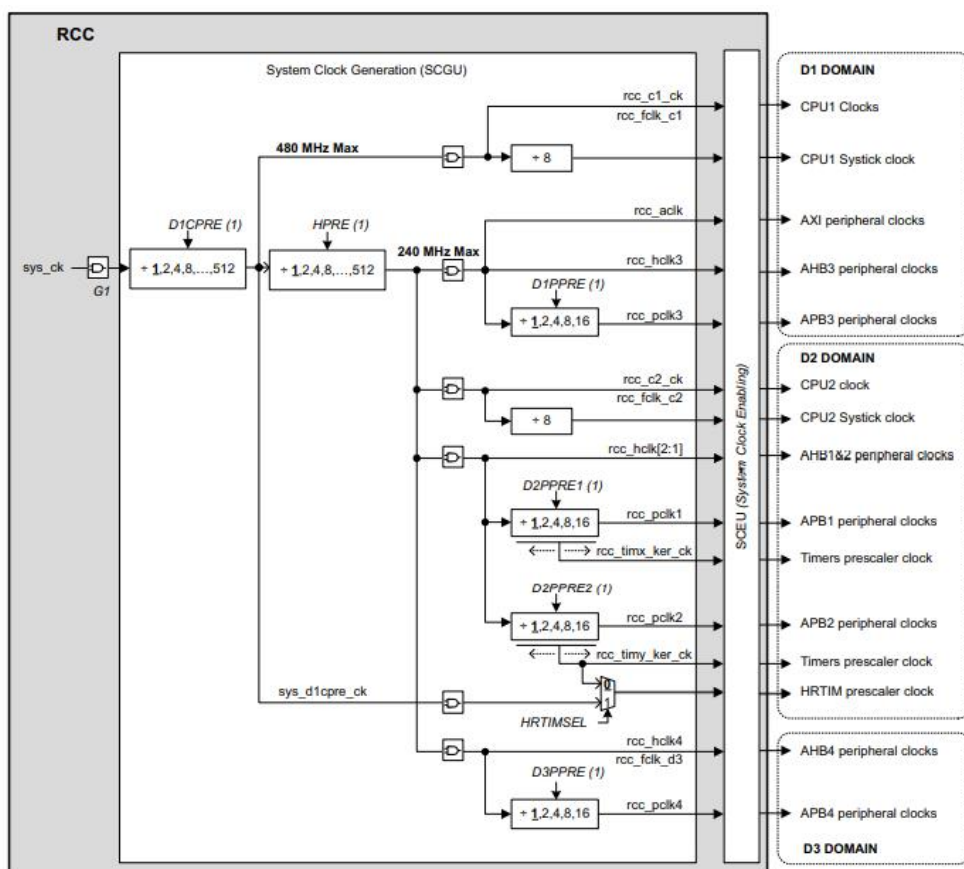


图4.2 STM32H747微控制器时钟系统

从上图可以归纳出三种重要的设备时钟：①Fclk——供给CPU内核的时钟信号，且一般等于系统时钟SYSclk；②Hclk——为优秀高性能总线（Advanced High-performance Bus, AHB）提供时钟信号，最主要的角色为RAM、Flash等外设的高速时钟，数值上也一般不进行分频，直接设置为SYSclk；③Pclk——为优秀高性能外设总线（Advanced Peripherals Bus）提供时钟信号，这些外设一般为UART, GPIO、PWM 定时器等。至此可知，这些设备时钟都是由系统时钟输出

直接或进行分频生成，并进一步成为设备各模块（如CPU、RAM等）的输入时钟。同时，STM32H7系列微控制器具有Cortex-M4/M7双核CPU，设备时钟频率上限等于相对较小的Cortex-M4频率上限（为Cortex-M7频率上限的1/2）。总而言之，通过进行不同的系统时钟配置，可以影响平台应用运行的计算、数据传输和存储等表现。

（2）电压配置

STM32H747微控制器的 V_{core} 是为平台上几乎所有的数字电路提供电压的核心域电源（Core Domain Supply），其中的D1域为Cortex-M7CPU（本课题智能应用运行使用的CPU核）和Flash的电压核区域。最重要的是，在运行模式下（Run Mode），微控制器提供了电压调整接口，可以用于获取不同的平台和应用性能表现。值得注意的是，平台的电压调整虽然能够影响应用的运行表现，但是必须和系统时钟频率进行同步配置。换言之，当前的系统时钟频率决定了微控制器的电压调整范围，只有当系统时钟频率低到一定阈值时，平台的电压才能够进行调整。本课题所使用的STM32H747微控制器提供了运行模式下的四种可调控电压，且其与时钟频率的关系如表4-1所示。

表4-1 STM32H747微控制器电压、时钟频率关系

电压级别	电压范围	系统时钟频率	设备时钟频率
级别1	1.26V~1.40V	480MHz Max	240MHz Max
级别2	1.15V~1.26V	400MHz Max	200MHz Max
级别3	1.05V~1.15V	300MHz Max	150MHz Max
级别4	0.95V~1.05V	200MHz Max	100MHz Max

综上，通过微控制器电压、系统时钟频率的联合配置可以调整平台性能，从而影响应用的模型运行时延和能耗等。本课题最终选取了多对不同的电压、频率配置，并结合基于压缩算子群的模型装载方案，实现了软硬协同的自适应策略搜索空间设计。由于两个策略空间的设计具备独立的流程，不存在冲突和耦合，令压缩算子群策略空间 A 中的策略数为 N_A ，电压频率调整策略空间 S 中的策略数为 N_S ，本课题软硬协同策略空间的搜索维度则为 $N_A * N_S$ 。这可以说明，本课题软硬协同设计的情境自适应框架，相比于单独模型或平台层面的设计更具多样性，能够更多维地实现情境的最优适配。

4.3 情境自适应框架实例验证

在本章实验的设计验证中，本课题选取了4种不同的电压、频率配置，作为硬件平台层面离散的策略搜索空间，如表4-2所示。

表4-2 本课题实际使用的电压、频率配置

配置	电压级别	系统时钟频率
配置1	级别1	480 MHz
配置2	级别2	400 MHz
配置3	级别3	300 MHz
配置4	级别4	200 MHz

在应用模型层面，本课题结合第三章基于压缩算子群的模型装载空间对本章的软硬协同框架进行实例验证。具体的，以“Bottleneck_75%”的压缩算子装载模型为例，其于不同电压、频率配置下的餐品分类应用表现如表4-3所示。

表4-3 Bottleneck_75%装载模型于不同电压、频率配置下的应用表现

配置	时延（ms）	能耗（W）
配置1	270	1.893
配置2	324	1.861
配置3	436	1.770
配置4	655	1.746

上表中，本课题的“能耗”指标通过如下设置进行测量：使用额定容量为6000mAh的E10000+爱国者充电宝作为STM32H747微控制器的外部供电电源。在餐品分类应用持续运行时，对外部电源每降低10%电量的所需用时 t 进行记录，并最终通过损失能量 W 、用时 t 以及式(4-10)计算出智能应用的运行能耗 P 。

$$P = \frac{W}{t} \quad (4-10)$$

值得注意的是，智能应用运行时不仅包含神经网络推理过程中计算和存储交换的能量消耗，还包括LCD或者可选摄像头等外设带来的消耗。但从表中依然可以看出，随着电压、频率配置的“降低”，模型的运行时延逐渐增长，所处应用的运行能耗也在不断降低。实验实例如图4.3所示。



图4.3 配置2下Bottleneck_75%装载模型实例展示

上图中，微控制器LCD左上角展示了压缩算子名称、模型总参数量（Paras）、当前系统时钟频率（Sysclock）以及单样例运行时延（Inference）。图4.4展示了通过摄像头获取图像，在配置2下的“深度可分离卷积块”装载模型的餐品分类应用实例。图中识别对象为巧克力蛋糕，模型运行时延为238ms。



图4.4 摄像头取帧配置2下的深度可分离卷积块装载模型实例展示

通过上述实例验证可观察到，不同压缩算子装载的模型及其所处应用，在微控制器平台不同的电压、频率配置之下呈现出了不同的时延和能耗表现。平台不同的电压、频率配置对深度模型在微控制器上的运行性能、资源开销造成了不同程度的影响。结合基于压缩算子群的模型装载框架，软硬协同的策略搜索空间为智能应用的情境自适应带来了更为丰富的多样性。

第五章 总结与展望

随着深度模型在各个领域逐渐达成了超越人类的水平，模型本身的时间复杂度和空间复杂度也不断地突破上限，对运行设备的计算、存储等能力提出了越来越高的要求。一方面，仅依靠于追求硬件设备性能和资源上限的不断拓展是不符合实际的。另一方面，迎合普适计算、万物互联的时代需求，相比于在个人电脑或云服务器等传统CPU/GPU计算平台，在资源受限的移动或嵌入式等物联网终端部署深度神经网络，进行设备智能的增效，缩短智能应用与用户之间的距离从而更好地服务于用户，其本身更是一种至关重要的未来趋势但也更具挑战。其中，微控制器以其小体积、低成本、低能耗等优势成为未来智能应用最为关键的部署设备之一。但其也因更加苛刻的极端资源条件（普遍MB级别以下的内存），极为显著地增加了深度模型在其上部署和运行的难度。此外，另一项挑战是在实际应用中，这些计算平台都处于动态变化的平台资源情境中（如电量、存储资源），加之用户需求的因素，对应用加载的模型有着不同层面的约束和需求（如能耗、时延）。然而，传统神经网络往往基于某种特定数据集和固定的拓扑结构进行训练生成，缺乏一定的动态机制，从而更不具备自适应调优的能力。

综上，在微控制器设备上成功部署深度模型，并使其具备自适应调优的能力，实现根据情境约束和需求进行精度和消耗的权衡折中和对情境的最优适配，对于终端智能应用的发展具有重要意义。

针对上述挑战，本课题主要研究了深度模型于微控制器平台的部署与情境自适应问题。（1）首先，本课题提出一种基于压缩算子群的轻量化模型结构和压缩方案设计，利用全局平均池化、模型结构同比例缩放思想等生成适配微控制器资源约束（如运行内存和持久性存储）的轻量化变体AlexNet模型。在此基础上，利用“结构块”思想构建出基于压缩算子群的模型装载框架，进一步使用模型量化压缩技术完成本体模型和不同配置压缩算子于微控制器平台的成功部署。（2）为了进一步完成终端智能应用的情境自适应，本课题提出一种软硬协同的情境自适应框架，从应用模型层面的压缩算子群装载设计和硬件平台层面的电压、频率调整设计两个层面构建起策略搜索空间，使得模型所装载的智能应用从软、硬件维度进行运行方案选择，使得策略搜索空间更加多样化。

实验结果表明，本课题在轻量化变体AlexNet的基础上和餐品分类任务中实现了50KB级别模型的生成和部署，在模型尺寸和时延的大幅缩减下，精度平均损失约为3%。同时，软硬协同的情境自适应框架也具有相比于单独模型或平台层面设计更多样的策略搜索空间，且具有很高的可拓展性。

在未来展望上，本工作可以改进的地方还有很多：首先，在资源受限的计算平台上进行深度模型的高效部署本身就涉及到软硬件层次的不同方面。在算法模型层，还可以有比本课题所涉及的轻量化变体AlexNet更合理有效的细粒度人工设计（比如结合底层效率），或利用平台资源自适应NAS生成更适应于当前平台的网络结构；在推理框架上，能够加速和优化模型在特定平台上推理的自定义设计可以被加入到流程中，例如使用TensorFlow-Lite、 μ TVM等工具；在底层/硬件平台层面，除电压频率调整技术外，一些更接近底层库支持或电路设计的神经网络推理优化也已存在众多相关研究工作。同样的，在深度模型的部署、推理优化之外，使其能于情境中更合理地选择精度与开销的情境自适应框架设计也是一项值得探索和改进的方向。其中最关键的问题就是如何建立起更加合理、有效的动态机制（压缩算子群、电压频率调整技术等）以及情境与决策之间的映射机制（情境感知的对象、决策空间的建立、感知指标与决策输出的关系（搜索策略）等）。还有，像能耗等真实平台性能或开销的指标获取方法或等效替代方案也是一项很实际且具有挑战性的工作。

参考文献

- [1] Deng L, Li J, Huang J T, et al. Recent advances in deep learning for speech research at Microsoft[C]//2013 IEEE International Conference on Acoustics, Speech and Signal Processing. IEEE, 2013: 8604-8608.
- [2] Krizhevsky A, Sutskever I, Hinton G E. Imagenet classification with deep convolutional neural networks[J]. Advances in neural information processing systems, 2012, 25: 1097-1105.
- [3] Collobert R, Weston J, Bottou L, et al. Natural language processing (almost) from scratch[J]. Journal of machine learning research, 2011, 12(ARTICLE): 2493–2537.
- [4] Chen C, Seff A, Kornhauser A, et al. Deepdriving: Learning affordance for direct perception in autonomous driving[C]//Proceedings of the IEEE international conference on computer vision. 2015: 2722-2730.
- [5] Esteva A, Kuprel B, Novoa R A, et al. Dermatologist-level classification of skin cancer with deep neural networks[J]. nature, 2017, 542(7639): 115-118.
- [6] Silver D, Huang A, Maddison C J, et al. Mastering the game of Go with deep neural networks and tree search[J]. nature, 2016, 529(7587): 484-489.
- [7] Simonyan K, Zisserman A. Very deep convolutional networks for large-scale image recognition[J]. arXiv preprint arXiv:1409.1556, 2014.
- [8] Augarten, Stan. The Most Widely Used Computer on a Chip: The TMS 1000. State of the Art: A Photographic History of the Integrated Circuit. New Haven and New York: Ticknor & Fields. ISBN 978-0-89919-195-9. Retrieved 2009-12-23.
- [9] Lin J, Chen W M, Lin Y, et al. Mcunet: Tiny deep learning on iot devices[J]. arXiv preprint arXiv:2007.10319, 2020.
- [10] Guo Y, Yao A, Chen Y. Dynamic network surgery for efficient dnns[J]. arXiv preprint arXiv:1608.04493, 2016.
- [11] Luo J H, Wu J, Lin W. Thinet: A filter level pruning method for deep neural network compression[C]//Proceedings of the IEEE international conference on computer vision. 2017: 5058-5066.
- [12] He Y, Zhang X, Sun J. Channel pruning for accelerating very deep neural

- networks[C]//Proceedings of the IEEE International Conference on Computer Vision. 2017: 1389-1397.
- [13]Gupta S, Agrawal A, Gopalakrishnan K, et al. Deep learning with limited numerical precision[C]//International conference on machine learning. PMLR, 2015: 1737-1746.
- [14]Vanhoucke V, Senior A, Mao M Z. Improving the speed of neural networks on CPUs[J]. 2011.
- [15]Courbariaux M, Bengio Y, David J P. Binaryconnect: Training deep neural networks with binary weights during propagations[J]. arXiv preprint arXiv:1511.00363, 2015.
- [16]Rastegari M, Ordonez V, Redmon J, et al. Xnor-net: Imagenet classification using binary convolutional neural networks[C]//European conference on computer vision. Springer, Cham, 2016: 525-542.
- [17]Jaderberg M, Vedaldi A, Zisserman A. Speeding up convolutional neural networks with low rank expansions[J]. arXiv preprint arXiv:1405.3866, 2014.
- [18]Zhang X, Zou J, He K, et al. Accelerating very deep convolutional networks for classification and detection[J]. IEEE transactions on pattern analysis and machine intelligence, 2015, 38(10): 1943-1955.
- [19]Lebedev V, Ganin Y, Rakhuba M, et al. Speeding-up convolutional neural networks using fine-tuned cp-decomposition[J]. arXiv preprint arXiv:1412.6553, 2014.
- [20]Romero A, Ballas N, Kahou S E, et al. Fitnets: Hints for thin deep nets[J]. arXiv preprint arXiv:1412.6550, 2014.
- [21]Yang T J, Chen Y H, Sze V. Designing energy-efficient convolutional neural networks using energy-aware pruning[C]//Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition. 2017: 5687-5695.
- [22]He K, Zhang X, Ren S, et al. Deep residual learning for image recognition[C]//Proceedings of the IEEE conference on computer vision and pattern recognition. 2016: 770-778.
- [23]Iandola F N, Han S, Moskewicz M W, et al. SqueezeNet: AlexNet-level accuracy with 50x fewer parameters and< 0.5 MB model size[J]. arXiv preprint arXiv:1602.07360, 2016.
- [24]Howard A G, Zhu M, Chen B, et al. Mobilenets: Efficient convolutional neural networks for mobile vision applications[J]. arXiv preprint arXiv:1704.04861, 2017.

- [25]Zhang X, Zhou X, Lin M, et al. Shufflenet: An extremely efficient convolutional neural network for mobile devices[C]//Proceedings of the IEEE conference on computer vision and pattern recognition. 2018: 6848-6856.
- [26]Zoph B, Vasudevan V, Shlens J, et al. Learning transferable architectures for scalable image recognition[C]//Proceedings of the IEEE conference on computer vision and pattern recognition. 2018: 8697-8710.
- [27]Real E, Aggarwal A, Huang Y, et al. Regularized evolution for image classifier architecture search[C]//Proceedings of the aaai conference on artificial intelligence. 2019, 33(01): 4780-4789.
- [28]Cai H, Yang J, Zhang W, et al. Path-level network transformation for efficient architecture search[C]//International Conference on Machine Learning. PMLR, 2018: 678-687.
- [29]Cai H, Zhu L, Han S. Proxylessnas: Direct neural architecture search on target task and hardware[J]. arXiv preprint arXiv:1812.00332, 2018.
- [30]Tan M, Chen B, Pang R, et al. Mnasnet: Platform-aware neural architecture search for mobile[C]//Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition. 2019: 2820-2828.
- [31]Wu B, Dai X, Zhang P, et al. Fbnet: Hardware-aware efficient convnet design via differentiable neural architecture search[C]//Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition. 2019: 10734-10742.
- [32]Fedorov I, Adams R P, Mattina M, et al. Sparse: Sparse architecture search for cnns on resource-constrained microcontrollers[J]. arXiv preprint arXiv:1905.12107, 2019.
- [33]Liberis E, Dudziak Ł, Lane N D. μ NAS: Constrained Neural Architecture Search for Microcontrollers[C]//Proceedings of the 1st Workshop on Machine Learning and Systems. 2021: 70-79.
- [34]Jiang W, Zhang X, Sha E H M, et al. Accuracy vs. efficiency: Achieving both through fpga-implementation aware neural architecture search[C]//Proceedings of the 56th Annual Design Automation Conference 2019. 2019: 1-6.
- [35]Hao C, Zhang X, Li Y, et al. Fpga/dnn co-design: An efficient design methodology for 1ot intelligence on the edge[C]//2019 56th ACM/IEEE Design Automation Conference (DAC). IEEE, 2019: 1-6.
- [36]Liu L, Deng J. Dynamic deep neural networks: Optimizing accuracy-efficiency trade-offs by selective execution[C]//Proceedings of the AAAI Conference on Artificial Intelligence. 2018, 32(1).

- [37]Wu Z, Nagarajan T, Kumar A, et al. Blockdrop: Dynamic inference paths in residual networks[C]//Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition. 2018: 8817-8826.
- [38]Teerapittayanon S, McDanel B, Kung H T. Branchynet: Fast inference via early exiting from deep neural networks[C]//2016 23rd International Conference on Pattern Recognition (ICPR). IEEE, 2016: 2464-2469.
- [39]Yu J, Yang L, Xu N, et al. Slimmable neural networks[J]. arXiv preprint arXiv:1812.08928, 2018.
- [40]Han Y, Huang G, Song S, et al. Dynamic neural networks: A survey[J]. arXiv preprint arXiv:2102.04906, 2021.
- [41]Sandler M, Howard A, Zhu M, et al. Mobilenetv2: Inverted residuals and linear bottlenecks[C]//Proceedings of the IEEE conference on computer vision and pattern recognition. 2018: 4510-4520.
- [42]David R, Duke J, Jain A, et al. Tensorflow lite micro: Embedded machine learning on tinymml systems[J]. arXiv preprint arXiv:2010.08678, 2020.
- [43]Martin C K, Correa J B, Han H, et al. Validity of the Remote Food Photography Method (RFPM) for estimating energy and nutrient intake in near real - time[J]. Obesity, 2012, 20(4): 891-899.
- [44]Bossard L, Guillaumin M, Van Gool L. Food-101—mining discriminative components with random forests[C]//European conference on computer vision. Springer, Cham, 2014: 446-461.
- [45]Glorot X, Bordes A, Bengio Y. Deep sparse rectifier neural networks[C]//Proceedings of the fourteenth international conference on artificial intelligence and statistics. JMLR Workshop and Conference Proceedings, 2011: 315-323.
- [46]Yang T J, Chen Y H, Sze V. Designing energy-efficient convolutional neural networks using energy-aware pruning[C]//Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition. 2017: 5687-5695.
- [47]Lin M, Chen Q, Yan S. Network in network[J]. arXiv preprint arXiv:1312.4400, 2013.
- [48]Hinton G E, Srivastava N, Krizhevsky A, et al. Improving neural networks by preventing co-adaptation of feature detectors[J]. arXiv preprint arXiv:1207.0580, 2012.
- [49]Ioffe S, Szegedy C. Batch normalization: Accelerating deep network training by reducing internal covariate shift[C]//International conference on machine learning.

- PMLR, 2015: 448-456.
- [50] Qin Z, Zhang Z, Chen X, et al. Fd-mobilenet: Improved mobilenet with a fast downsampling strategy[C]//2018 25th IEEE International Conference on Image Processing (ICIP). IEEE, 2018: 1363-1367.
- [51] Szegedy C, Vanhoucke V, Ioffe S, et al. Rethinking the inception architecture for computer vision[C]//Proceedings of the IEEE conference on computer vision and pattern recognition. 2016: 2818-2826.
- [52] Wu J, Wang Y, Wu Z, et al. Deep k-means: Re-training and parameter sharing with harder cluster assignments for compressing deep convolutions[C]//International Conference on Machine Learning. PMLR, 2018: 5363-5372.
- [53] Jacob B, Kligys S, Chen B, et al. Quantization and training of neural networks for efficient integer-arithmetic-only inference[C]//Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition. 2018: 2704-2713.
- [54] Williams S. Roofline: An Insightful Visual Performance Model for Floating-Point Programs and Multicore[J]. ACM Communications, 2009.
- [55] 卢春鹏. 动态电压与频率调节在降低功耗中的作用[J]. 单片机与嵌入式系统应用, 2007 (5): 12-14.

致 谢

时光飞逝，大学生活在忙忙碌碌的日子里悄悄迎来了尾声。我的本科四年拥有着太多值得回忆的事情，也有着太多值得纪念和感谢的人。

首先，我想对郭斌教授致以感谢。在我们每一位同学都要为人生的下一阶段进行选择时，是您给了我进入实验室大家庭的钥匙，更为我指引和提供了更多的可能性。进入实验室的大半年，您对学术的严谨务实和宽阔视野、对学生的细致关怀还有对事情的认真态度都总让我感受到自己的幸运，并以您为榜样受到鼓舞。当然，我还要感谢我的毕设指导老师刘思聪老师，不仅在我的学术方向上给予我宝贵和难得的知识，还从生活和经验上给予我关怀、鼓励和引导，您对学术纯粹的喜爱和专研成为我努力学习的方向。

同时，我要感谢实验室的王虹力师姐、王豪师兄、仵云港师兄、张秋韵师姐和122实验室的等等师兄师姐们。在我刚进入实验室时，你们的友好和关怀帮助我渐渐习惯和适应。平常你们是和蔼可亲的哥哥姐姐；科研时你们是能力出众、乐于解答的榜样。你们的存在让我更好地进行在实验室的生活和学习。

另外，我还想对大学四年里相识和陪伴着我的好朋友们说一声感谢。你们给我的力量是最难以总结和记录的。也是平常最经常接触的你们，组成了我最多元、最丰富的大学记忆。我们身边的人总是来来去去，但这本回忆录里却会永远记载着我们的瞬间。未来的路充满着未知和神秘，希望在人生路上每一个重要节点，我们回望过去都可以发现自己的成长，并顺便记起种种美好。也愿我们永远年轻，在未来也许相见的交叉路口虽然风尘仆仆但依然满面春光。

此外，我要感谢我的家人们对我的爱与支持，更要感谢我生命里最重要的人——母亲。您总说拥有我是您的幸运，但其实能成为您的儿子才是我一生中最为幸运的“决定”。不管是命运还是选择，都让我们紧紧相连，我最希望的就是能和您一起走向更美好的未来。

最后感谢评阅老师的用心评阅。

毕业设计小结

至此，我的本科毕业设计也要画上圆满的句号了。在这半年的学习和探索中，不仅是在知识层面还是在个人经历层面，我都收获颇丰。

首先，作为我本科生涯的最后一个环节，毕设体现了我在本科所积累的知识 and 能力。进入实验室的大半年以来，在老师和师兄师姐的指导下，我慢慢地走进了科研的世界。从文献阅读到知识归纳，从问题抽象到建模分析与代码实践，每一个过程都是在为后面的道路做铺垫。但其中每一个过程也都属实不易，需要投入大量的精力和专注力。“脚踏实地”、“厚积薄发”是我在毕设期间对科研工作的最大感受。

同时，作为在本科生涯最后送给自己的礼物，毕设装载了我的思考和对大学生活的回忆。时光如白马过隙，在本科迷茫和求解的路上，我们每个人都尝试过大大小小的选择。对我来说，其中最重要的可能就是自己在某一个时间点跳出了原来的舒适圈，在提升自己的过程中同时遇到了许多优秀的朋友，一起努力、一起成长。在未来的某一天，我一定会回望这些过往以及它们最后的答卷——毕设，其承载的不仅是我对科研的初步尝试，更记录了我的人生道路上的又一里程碑。

在大学的最后阶段，我们总是忙于处理最后的要紧事，忙于奔波各自的未来或是总结回忆起种种曾经和成长。在这个小结里，我也顺便记录一下我的本科生涯，希望我们未来都能顺利抵达自己的Wonderland，成为自己想要成为的人。